



DOCUMENTO 10

Documentación Final

Documento maestro del proyecto PARKINLOCK

Proyecto	PARKINLOCK — Sistema de gestión de parqueadero
Institución	I.E.R. Santa María
Integrantes	Loren
Docente	Liseth Giraldo
Versión / Fecha	1.0 — 1 de octubre de 2026

Contenido

1. Presentación y estado de la información

2. Descripción general

3. Requisitos funcionales

4. Requisitos no funcionales

5. Historias de usuario

6. Modelo entidad-relación

7. Base de datos MySQL

8. Arquitectura del sistema

9. Tecnologías y documentación técnica

10. Manual de usuario del diseño funcional

11. Estado actual y trabajo pendiente

12. Conclusiones

1. Presentación y estado de la información

Fuentes y estado de la información

Esta documentación se elaboró con tres fuentes: el **prototipo de Figma** del sistema (11 pantallas), el **código del repositorio** PARKINLOCK y sus archivos de configuración, y el documento de **requisitos funcionales** derivado del prototipo. Cada elemento se marca así:

- **EXISTENTE EN CÓDIGO** Verificado en el código actual del repositorio.
- **DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN** Aparece en el prototipo de Figma, pero no existe en el código.
- **PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN** Diseño técnico planteado en esta documentación para soportar lo diseñado en Figma.

Estado real: 21 funcionalidades diseñadas en Figma, **0 implementadas** en el código, 21 pendientes de implementación.

2. Descripción general

2.1 Ficha del proyecto

Campo	Información
Nombre oficial	PARKINLOCK
Identidad	Parqueadero Systems & Programación
Tipo de aplicación	Aplicación web progresiva (PWA) para la gestión de un parqueadero
Institución	I.E.R. Santa María
Integrantes	Loren
Docente	Liseth Giraldo
Repositorio	github.com/camilotobon34-sudo/Parkinloock-
Diseño funcional	Prototipo en Figma (11 pantallas: 7 de escritorio y 4 de móvil)
Versión del documento	1.0 — 1 de octubre de 2026

2.2 Descripción del proyecto

PARKINLOCK es un sistema web para la administración de un parqueadero, construido como aplicación web progresiva (PWA). Según el archivo README del repositorio, el parqueadero es un *"espacio destinado al estacionamiento organizado y seguro de vehículos de estudiantes, profesores y personal de la institución, facilitando el control y uso adecuado de los espacios disponibles"*.

El prototipo de Figma define cómo funcionará el sistema: los usuarios ingresan con el rol de **Administrador** o **Trabajador**; registran la entrada de carros y motos con su placa y espacio; consultan en un mapa el estado de cada espacio (disponible, ocupado o reservado); calculan el tiempo de permanencia y el valor a pagar según las tarifas; reciben el pago en efectivo, calculan el cambio y registran la salida liberando el espacio. Además, contempla reservas, mensualidades, historial de movimientos, configuración de tarifas, permisos por rol y una versión para teléfono móvil.

Modelo de cobro

El uso del parqueadero se cobra según las tarifas configuradas por el Administrador, tal como lo define el prototipo de Figma y lo indica el README: cobro por tiempo (hora, fracción, noche y día), cargo por reserva y mensualidades, con pago en efectivo. Las reglas de cálculo (RC-01 a RC-10) están en el documento 02.

2.3 Planteamiento del problema

Sin un sistema de información, el control de un parqueadero depende de registros manuales, lo que dificulta:

- Conocer en tiempo real qué espacios están disponibles, ocupados o reservados.
- Registrar con exactitud la hora de entrada y salida de cada vehículo y calcular correctamente el tiempo y el valor a cobrar.
- Llevar el control del dinero recibido, el cambio entregado y lo recaudado en cada turno.
- Administrar reservas y mensualidades y saber cuáles están activas o vencidas.
- Consultar el historial de movimientos y restringir funciones según el rol de cada persona.

Datos cuantitativos del parqueadero de la institución (número de espacios, vehículos diarios, forma actual de control): **[Pendiente de completar]**

2.4 Justificación

Un sistema como PARKINLOCK permite organizar el uso de los espacios, reducir errores en los cobros, agilizar la atención en la entrada y la salida, y conservar un historial confiable de la operación. El prototipo ya define las pantallas para escritorio y móvil, lo que facilita su uso tanto desde un computador como desde un teléfono. Construirlo como PWA permite instalarlo en el dispositivo sin depender de una tienda de aplicaciones.

2.5 Objetivo general

Desarrollar PARKINLOCK, una aplicación web progresiva que permita gestionar las entradas, salidas, cobros, espacios, reservas y mensualidades de un parqueadero, con acceso diferenciado para Administrador y Trabajador, de acuerdo con el diseño funcional elaborado en Figma.

2.6 Objetivos específicos

1. Implementar el inicio de sesión con selección de rol y el control de acceso según la matriz de permisos (RF-001 a RF-004).
2. Construir el panel con indicadores, mapa de espacios y últimos movimientos (RF-005 a RF-007).
3. Implementar el flujo operativo de entrada, búsqueda por placa, cálculo del cobro, pago con cambio y salida con liberación del espacio (RF-008 a RF-012).
4. Implementar la gestión de reservas, mensualidades e historial con exportación (RF-013 a RF-016).
5. Permitir al Administrador configurar tarifas y gestionar usuarios (RF-017 y RF-018).
6. Incorporar la consulta de vehículos en operación, las alertas operativas y el turno activo (RF-019 a RF-021).

2.7 Alcance

Incluido (según el prototipo de Figma)	No incluido o no definido en el prototipo
Inicio de sesión por rol y control de acceso	Pagos electrónicos (el prototipo solo muestra pago en efectivo)
Panel, indicadores y mapa de espacios	Lectura automática de placas o control de barreras
Registro de entrada y salida con cobro	Integración con sensores (aparece una alerta de sensor, pero no su funcionamiento)
Reservas y mensualidades	Gestión de varias sedes
Historial y exportación	Pantallas de Vehículos, Clientes y Reportes (solo aparecen en un menú)
Tarifas, permisos y gestión de usuarios	Flujo completo de recuperación de contraseña
Versión móvil (inicio de sesión, panel, entrada y salida)	—

2.8 Usuarios del sistema

Rol	Descripción	Permisos principales (matriz de Figma)
Administrador	Responsable de la sede. Configura el sistema y supervisa la operación.	Ver y editar panel; gestionar entradas, salidas, reservas y mensualidades; ver y exportar historial; configurar tarifas y usuarios.
Trabajador	Operador de turno que atiende la entrada y la salida de vehículos.	Ver panel; gestionar entradas y salidas; crear y ver reservas; solo ver mensualidades; ver historial de su turno; sin acceso a tarifas y usuarios.

2.9 Beneficios esperados

- Visibilidad en tiempo real de la ocupación del parqueadero.
- Cobros calculados automáticamente según tarifas configurables.

- Cálculo del cambio y aviso cuando el dinero recibido no alcanza.
- Historial de entradas, salidas y pagos por trabajador.
- Separación de funciones entre Administrador y Trabajador.
- Uso desde computador o teléfono, con posibilidad de instalar la aplicación.

2.10 Contexto del sistema

Elemento	Relación con PARKINLOCK
Administrador	Configura tarifas, usuarios y permisos; consulta indicadores, historial y exporta información.
Trabajador	Registra entradas y salidas, cobra, crea reservas y consulta el mapa de espacios.
Cliente / conductor	No usa el sistema directamente; sus datos (placa, nombre) se registran en entradas, reservas y mensualidades.
Vehículo	Carro o moto identificado por su placa.
Espacio	Lugar de estacionamiento identificado por un código (por ejemplo, C-12 para carro o M-05 para moto).

2.11 Estado actual del proyecto

Componente	Estado	Detalle verificado
Diseño funcional (Figma)	COMPLETO	11 pantallas que cubren 21 funcionalidades (RF-001 a RF-021).
Frontend	EXISTENTE EN CÓDIGO	Plantilla base de Vite + React con configuración PWA (manifest, service worker e íconos). No contiene pantallas del parqueadero.
Pantallas de PARKINLOCK	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN	Ninguna de las 11 pantallas está construida en el código.
Backend / API	NO INICIADO	La carpeta backend-Parkinlock está vacía y no está registrada en Git.
Base de datos	PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN	No existe en el código. Se propone un modelo MySQL (documentos 05 y 06).
Despliegue	NO REALIZADO	No hay evidencia de despliegue.

Indicador	Cantidad
Funcionalidades diseñadas en Figma	21
Funcionalidades implementadas en código	0
Funcionalidades pendientes de implementación	21

3. Requisitos funcionales

3.1 Clasificación de las funcionalidades

Categoría	Cantidad	Detalle
Diseñadas en Figma	21	RF-001 a RF-021.
Implementadas en el código	0	El código solo contiene la plantilla de Vite y la configuración PWA, que corresponden a requisitos no funcionales.
Pendientes de implementación	21	Todas las funcionalidades diseñadas.

3.2 Pantallas del prototipo analizadas

ID	Pantalla	Elementos visibles
F01	Inicio de sesión (escritorio)	Selector de rol Administrador/Trabajador, correo o usuario, contraseña con botón para mostrarla, "Recordar acceso", "¿Olvidaste tu clave?", botón "Entrar al sistema".
F02	Panel general (escritorio, rol Administrador)	Indicadores, botones "Registrar entrada", "Cobrar salida" y "Nueva reserva", mapa de 48 espacios, ocupación actual, últimos movimientos, turno activo y menú lateral completo.
F03	Panel general – variante con logo Parquadero	Indicadores del día, espacios en tiempo real por nivel, alertas operativas, actividad reciente, tabla "Vehículos en operación" con búsqueda, filtro y paginación, notificaciones. Menú: Entradas, Salidas y cobros, Reservas, Vehículos, Clientes, Reportes.
F04	Registrar entrada (escritorio, rol Trabajador)	Secuencia de 7 pasos, tipo de vehículo Carro/Moto, placa, espacio, hora automática, validación de disponibilidad, tarjeta del registro, espacios cercanos disponibles, botones "Cancelar" y "Guardar placa y espacio".
F05	Salida y cobro (escritorio)	Búsqueda por placa, datos de entrada/salida/tiempo, cálculo del cobro por hora y fracción, recibir pago en efectivo, cambio, aviso de monto insuficiente, botón "Registrar salida y liberar C-12".
F06	Gestión de clientes (escritorio)	Pestañas Reservas, Mensualidades e Historial; "Nueva reserva", "Nueva mensualidad", historial de movimientos con búsqueda por placa, filtro de fecha y "Exportar".
F07	Configuración (escritorio, exclusiva del Administrador)	Tarifas vigentes por concepto y tipo de vehículo, "Guardar cambios", matriz de permisos por rol y botón "Gestionar usuarios".
F08	Inicio de sesión (móvil)	Selector de rol, usuario, contraseña y botón "Entrar".
F09	Panel (móvil, rol Trabajador)	Indicadores, botones "Entrada" y "Cobrar", mapa rápido, "Ver todos", reserva próxima y barra inferior Panel/Entrada/Salida/Más.
F10	Registrar entrada (móvil)	Mismo formulario de entrada y secuencia operativa de 7 pasos.
F11	Salida y cobro (móvil)	Búsqueda por placa, total a pagar, recibir pago, cambio y botón "Registrar salida".

Observaciones

- Los roles definidos en Figma son **Administrador** y **Trabajador**.
- La pantalla F03 muestra en el menú "Vehículos", "Clientes" y "Reportes", sin pantallas asociadas; por eso no se documentan como requisitos.
- RF-002, RF-013, RF-014, RF-016 y RF-018 aparecen en Figma como enlace o botón, sin la pantalla de destino; sus detalles quedan pendientes.
- **Las prioridades son una propuesta** del equipo de documentación; Figma no define prioridades.

Algunas pantallas del prototipo muestran un nombre de marca provisional. El nombre oficial del proyecto es **PARKINLOCK**.

3.3 Tabla resumen de requisitos funcionales

Código	Requisito funcional	Actor	Prioridad	Evidencia
RF-001	Iniciar sesión con selección de rol	Administrador, Trabajador	Alta	F01, F08
RF-002	Recuperar contraseña	Administrador, Trabajador	Baja	F01
RF-003	Cerrar sesión	Administrador, Trabajador	Media	F02, F04, F05, F06, F07
RF-004	Control de acceso por rol	Sistema	Alta	F02, F04, F07
RF-005	Visualizar panel general de indicadores	Administrador, Trabajador	Media	F02, F03, F09
RF-006	Visualizar mapa de espacios	Administrador, Trabajador	Alta	F02, F03, F09
RF-007	Consultar últimos movimientos	Administrador, Trabajador	Media	F02, F03
RF-008	Registrar entrada de vehículo	Administrador, Trabajador	Alta	F04, F10
RF-009	Buscar vehículo por placa para la salida	Administrador, Trabajador	Alta	F05, F11
RF-010	Calcular tiempo y valor a cobrar	Sistema	Alta	F05, F07, F11
RF-011	Recibir pago y calcular cambio	Administrador, Trabajador	Alta	F05, F11
RF-012	Registrar salida y liberar espacio	Administrador, Trabajador	Alta	F04, F05, F10, F11
RF-013	Gestionar reservas	Administrador (gestionar), Trabajador (crear y ver)	Media	F02, F06, F09
RF-014	Gestionar mensualidades	Administrador (gestionar), Trabajador (solo ver)	Media	F06
RF-015	Consultar historial de movimientos	Administrador (todo), Trabajador (turno propio)	Media	F06
RF-016	Exportar historial	Administrador	Baja	F06, F07
RF-017	Configurar tarifas	Administrador	Alta	F07
RF-018	Gestionar usuarios	Administrador	Baja	F07
RF-019	Consultar vehículos en operación	Administrador, Trabajador	Media	F03

Código	Requisito funcional	Actor	Prioridad	Evidencia
RF-020	Mostrar alertas operativas	Administrador, Trabajador	Baja	F03
RF-021	Visualizar turno activo	Administrador, Trabajador	Baja	F02, F03, F04

3.4 Detalle de requisitos funcionales

3.4.1 RF-001 · Iniciar sesión con selección de rol

Campo	Detalle
Código	RF-001
Nombre	Iniciar sesión con selección de rol
Descripción	El sistema debe permitir el ingreso seleccionando el rol (Administrador o Trabajador) e ingresando correo o usuario y contraseña.
Actor	Administrador, Trabajador
Prioridad (propuesta)	Alta
Criterios de aceptación	1. El formulario muestra el selector de rol con las opciones "Administrador" y "Trabajador". 2. Solicita "Correo o usuario" y "Contraseña". 3. La contraseña se muestra oculta y puede hacerse visible con el ícono de ojo. 4. Incluye la opción "Recordar acceso". 5. Al pulsar "Entrar al sistema" (escritorio) o "Entrar" (móvil) se accede al panel del sistema.
Evidencia (Figma)	F01, F08
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

3.4.2 RF-002 · Recuperar contraseña

Campo	Detalle
Código	RF-002
Nombre	Recuperar contraseña
Descripción	El sistema debe ofrecer una opción para recuperar la clave desde la pantalla de inicio de sesión.
Actor	Administrador, Trabajador
Prioridad (propuesta)	Baja
Criterios de aceptación	1. La pantalla de inicio de sesión muestra el enlace "¿Olvidaste tu clave?". 2. Se muestra el mensaje "¿Necesitas ayuda? Contacta al administrador de la sede." 3. Pendiente: el prototipo no incluye la pantalla ni el flujo de recuperación.
Evidencia (Figma)	F01
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

3.4.3 RF-003 · Cerrar sesión

Campo	Detalle
Código	RF-003
Nombre	Cerrar sesión
Descripción	El sistema debe permitir al usuario finalizar su sesión desde el menú lateral.
Actor	Administrador, Trabajador
Prioridad (propuesta)	Media
Criterios de aceptación	1. El menú lateral muestra el nombre y rol del usuario autenticado (por ejemplo, "Andrea Mora · Administrador", "Carlos Silva · Trabajador"). 2. Junto al usuario se muestra un ícono de salida para cerrar la sesión. 3. Pendiente: el prototipo no muestra la pantalla de confirmación ni el destino tras cerrar sesión.
Evidencia (Figma)	F02, F04, F05, F06, F07
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

3.4.4 RF-004 · Control de acceso por rol

Campo	Detalle
Código	RF-004
Nombre	Control de acceso por rol
Descripción	El sistema debe mostrar y permitir las funciones según el rol del usuario, de acuerdo con la matriz de permisos.
Actor	Sistema
Prioridad (propuesta)	Alta
Criterios de aceptación	1. El Administrador ve en el menú la opción "Configuración"; el Trabajador no la ve. 2. Panel y mapa: Administrador "Ver y editar"; Trabajador "Ver". 3. Entradas y salidas: ambos roles "Gestionar". 4. Reservas: Administrador "Gestionar"; Trabajador "Crear y ver". 5. Mensualidades: Administrador "Gestionar"; Trabajador "Solo ver". 6. Historial: Administrador "Ver y exportar"; Trabajador "Ver turno propio". 7. Tarifas y usuarios: Administrador "Configurar"; Trabajador "Sin acceso".
Evidencia (Figma)	F02, F04, F07
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

3.4.5 RF-005 · Visualizar panel general de indicadores

Campo	Detalle
Código	RF-005
Nombre	Visualizar panel general de indicadores

Campo	Detalle
Descripción	El sistema debe mostrar un resumen operativo de la sede con los indicadores principales del parqueadero.
Actor	Administrador, Trabajador
Prioridad (propuesta)	Media
Criterios de aceptación	1. Muestra vehículos dentro (con desglose de carros y motos), espacios disponibles, ocupados y reservados, y el valor recaudado del día. 2. Muestra el porcentaje de ocupación actual y el pico esperado. 3. Ofrece accesos directos: "Registrar entrada", "Cobrar salida" y "Nueva reserva" (escritorio); "Entrada" y "Cobrar" (móvil). 4. Muestra la fecha y la hora actual.
Evidencia (Figma)	F02, F03, F09
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

3.4.6 RF-006 · Visualizar mapa de espacios

Campo	Detalle
Código	RF-006
Nombre	Visualizar mapa de espacios
Descripción	El sistema debe mostrar el mapa de espacios del parqueadero con su estado en tiempo real.
Actor	Administrador, Trabajador
Prioridad (propuesta)	Alta
Criterios de aceptación	1. Cada espacio muestra su código (C- para carros, M- para motos) y su estado: Libre/Disponible, Ocupado o Reserva. 2. Los espacios ocupados muestran la placa del vehículo. 3. Se muestra una leyenda con el total de espacios disponibles, ocupados y reservados. 4. En móvil se muestra un "Mapa rápido" con la opción "Ver todos".
Evidencia (Figma)	F02, F03, F09
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

3.4.7 RF-007 · Consultar últimos movimientos

Campo	Detalle
Código	RF-007
Nombre	Consultar últimos movimientos
Descripción	El sistema debe listar los movimientos más recientes del parqueadero en el panel.
Actor	Administrador, Trabajador
Prioridad (propuesta)	Media

Campo	Detalle
Criterios de aceptación	1. Cada movimiento muestra hora, placa, tipo de movimiento (entrada, salida con valor, en parqueadero), espacio y trabajador. 2. Incluye el enlace "Ver historial".
Evidencia (Figma)	F02, F03
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

3.4.8 RF-008 · Registrar entrada de vehículo

Campo	Detalle
Código	RF-008
Nombre	Registrar entrada de vehículo
Descripción	El sistema debe permitir registrar el ingreso de un vehículo asignándole placa y espacio.
Actor	Administrador, Trabajador
Prioridad (propuesta)	Alta
Criterios de aceptación	1. Permite seleccionar el tipo de vehículo: Carro o Moto. 2. Solicita la placa y valida su formato ("Formato validado · sin coincidencias activas"). 3. Permite seleccionar el espacio de una lista filtrada por tipo de vehículo ("Espacio para carro"). 4. La hora de entrada se registra automáticamente al confirmar. 5. Indica si el espacio está disponible ("C-12 está disponible · Se marcará como ocupado al guardar"). 6. Muestra los espacios cercanos disponibles y una tarjeta con placa, espacio, hora de entrada y tipo. 7. Botones "Cancelar" y "Guardar placa y espacio"; al guardar, el espacio pasa a ocupado.
Evidencia (Figma)	F04, F10
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

3.4.9 RF-009 · Buscar vehículo por placa para la salida

Campo	Detalle
Código	RF-009
Nombre	Buscar vehículo por placa para la salida
Descripción	El sistema debe permitir buscar un vehículo por su placa para iniciar el proceso de salida.
Actor	Administrador, Trabajador
Prioridad (propuesta)	Alta
Criterios de aceptación	1. Incluye el campo "Buscar vehículo por placa" y el botón "Buscar". 2. Si la placa existe, muestra el estado "Encontrado", el tipo de vehículo, la tarifa y el espacio ocupado. 3. Muestra la hora de entrada registrada, la hora de salida calculada y el tiempo utilizado.
Evidencia (Figma)	F05, F11

Campo	Detalle
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

3.4.10 RF-010 · Calcular tiempo y valor a cobrar

Campo	Detalle
Código	RF-010
Nombre	Calcular tiempo y valor a cobrar
Descripción	El sistema debe calcular el tiempo de permanencia y el valor a pagar según las tarifas vigentes.
Actor	Sistema
Prioridad (propuesta)	Alta
Criterios de aceptación	1. Calcula el tiempo utilizado entre la entrada y la salida (por ejemplo, "2 h 28 min"). 2. Muestra el desglose del cobro por hora y por fracción iniciada. 3. Muestra el "Total a pagar". 4. Usa las tarifas configuradas para el tipo de vehículo (ver RF-017). 5. Aplica las reglas de cobro RC-01 a RC-05 (tabla "Reglas de cobro").
Evidencia (Figma)	F05, F07, F11
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

3.4.11 RF-011 · Recibir pago y calcular cambio

Campo	Detalle
Código	RF-011
Nombre	Recibir pago y calcular cambio
Descripción	El sistema debe registrar el efectivo recibido y calcular el cambio para el cliente.
Actor	Administrador, Trabajador
Prioridad (propuesta)	Alta
Criterios de aceptación	1. Muestra el medio de pago "Efectivo" y el total. 2. Solicita el "Efectivo recibido". 3. Si el monto es suficiente, muestra "Monto suficiente" y el "Cambio para el cliente". 4. Si el monto no alcanza, muestra un aviso con el valor faltante (por ejemplo, "Con \$12.000, faltan \$3.000 para completar"). 5. El mismo registro de pago se usa para salidas, cargos de reserva y mensualidades (RC-06 a RC-08).
Evidencia (Figma)	F05, F11
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

3.4.12 RF-012 · Registrar salida y liberar espacio

Campo	Detalle
Código	RF-012
Nombre	Registrar salida y liberar espacio
Descripción	El sistema debe registrar la salida del vehículo y cambiar el estado del espacio a disponible.
Actor	Administrador, Trabajador
Prioridad (propuesta)	Alta
Criterios de aceptación	1. Antes de confirmar, muestra un resumen con hora de salida, pago, cambio y el cambio de estado del espacio (Ocupado → Disponible). 2. El botón "Registrar salida y liberar [espacio]" registra la salida. 3. Al confirmar, el espacio cambia de ocupado a disponible. 4. El proceso sigue la secuencia: Entrada, Placa y espacio, Calcular tiempo, Cobrar, Cambio, Registrar salida, Liberar espacio.
Evidencia (Figma)	F04, F05, F10, F11
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

3.4.13 RF-013 · Gestionar reservas

Campo	Detalle
Código	RF-013
Nombre	Gestionar reservas
Descripción	El sistema debe permitir consultar las reservas del día y crear nuevas reservas.
Actor	Administrador (gestionar), Trabajador (crear y ver)
Prioridad (propuesta)	Media
Criterios de aceptación	1. Muestra las reservas de hoy con placa, cliente, horario, espacio asignado y estado (Confirmada o Pendiente). 2. Muestra el resumen de reservas programadas y espacios aún sin asignar. 3. Incluye el botón "Nueva reserva" (también disponible en el panel). 4. El panel móvil muestra la reserva próxima. 5. Al confirmar la reserva se cobra el cargo por reserva del tipo de vehículo (RC-06, regla propuesta). 6. Pendiente: el prototipo no incluye el formulario de nueva reserva.
Evidencia (Figma)	F02, F06, F09
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

3.4.14 RF-014 · Gestionar mensualidades

Campo	Detalle
Código	RF-014
Nombre	Gestionar mensualidades

Campo	Detalle
Descripción	El sistema debe permitir consultar y crear planes de mensualidad para vehículos.
Actor	Administrador (gestionar), Trabajador (solo ver)
Prioridad (propuesta)	Media
Criterios de aceptación	1. Muestra la lista de mensualidades con placa, titular, plan (carro o moto mensual), vigencia, valor y estado (Activa o Vencida). 2. Muestra el resumen de planes activos, por vencer (5 días o menos para la fecha de fin, RC-10) y vencidos. 3. La vigencia de cada plan es de un mes desde la fecha de inicio, ambas fechas incluidas (por ejemplo, "01-31 oct" o "01-30 sep") (RC-07). 4. Incluye el botón "Nueva mensualidad". 5. El valor del plan es la tarifa de mensualidad vigente y su pago queda registrado (RC-07). 6. Un vehículo con mensualidad activa no paga por tiempo al salir (RC-05, regla propuesta). 7. Pendiente: el prototipo no incluye el formulario de nueva mensualidad.
Evidencia (Figma)	F06
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

3.4.15 RF-015 · Consultar historial de movimientos

Campo	Detalle
Código	RF-015
Nombre	Consultar historial de movimientos
Descripción	El sistema debe permitir consultar todas las entradas, salidas y pagos registrados.
Actor	Administrador (todo), Trabajador (turno propio)
Prioridad (propuesta)	Media
Criterios de aceptación	1. Muestra placa, fecha, hora de entrada, hora de salida, tiempo, valor pagado y trabajador. 2. Permite buscar por placa. 3. Permite filtrar por fecha.
Evidencia (Figma)	F06
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

3.4.16 RF-016 · Exportar historial

Campo	Detalle
Código	RF-016
Nombre	Exportar historial
Descripción	El sistema debe permitir exportar el historial de movimientos.
Actor	Administrador
Prioridad (propuesta)	Baja

Campo	Detalle
Criterios de aceptación	1. El historial incluye el botón "Exportar". 2. Solo el Administrador tiene el permiso "Ver y exportar". 3. Pendiente: el prototipo no define el formato del archivo exportado.
Evidencia (Figma)	F06, F07
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

3.4.17 RF-017 · Configurar tarifas

Campo	Detalle
Código	RF-017
Nombre	Configurar tarifas
Descripción	El sistema debe permitir al Administrador definir las tarifas usadas para calcular los cobros.
Actor	Administrador
Prioridad (propuesta)	Alta
Criterios de aceptación	1. Permite editar el valor para Carro y Moto de los conceptos: Hora (cada hora iniciada), Fracción (hasta 30 minutos), Noche (18:00–06:00), Día (máximo 12 horas), Reserva (cargo por reserva) y Mensualidad (vigencia de un mes, RC-07; la pantalla la rotula como "30 días calendario"). 2. Incluye el botón "Guardar cambios". 3. Muestra un ejemplo del cálculo con las tarifas vigentes. 4. La pantalla indica "Configuración exclusiva del administrador".
Evidencia (Figma)	F07
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

3.4.18 RF-018 · Gestionar usuarios

Campo	Detalle
Código	RF-018
Nombre	Gestionar usuarios
Descripción	El sistema debe permitir al Administrador gestionar los usuarios del sistema.
Actor	Administrador
Prioridad (propuesta)	Baja
Criterios de aceptación	1. La pantalla de Configuración incluye el botón "Gestionar usuarios". 2. Solo el Administrador tiene acceso ("Tarifas y usuarios: Configurar"). 3. Pendiente: el prototipo no incluye la pantalla de gestión de usuarios.
Evidencia (Figma)	F07
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

3.4.19 RF-019 · Consultar vehículos en operación

Campo	Detalle
Código	RF-019
Nombre	Consultar vehículos en operación
Descripción	El sistema debe listar los vehículos que se encuentran en operación en el parqueadero.
Actor	Administrador, Trabajador
Prioridad (propuesta)	Media
Criterios de aceptación	1. Muestra placa, cliente, tipo, hora de entrada, permanencia, espacio, estado (Estacionado, Por vencer, Reserva) y acciones. 2. Permite buscar por placa o cliente y filtrar. 3. Incluye paginación y la opción "Ver todos".
Evidencia (Figma)	F03
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

3.4.20 RF-020 · Mostrar alertas operativas

Campo	Detalle
Código	RF-020
Nombre	Mostrar alertas operativas
Descripción	El sistema debe mostrar alertas sobre situaciones que requieren atención.
Actor	Administrador, Trabajador
Prioridad (propuesta)	Baja
Criterios de aceptación	1. Muestra el número de alertas activas. 2. Ejemplos visibles: "Reserva A05 vence en 13 min · Cliente aún no registra ingreso" y "Sensor E12 sin respuesta". 3. Incluye un ícono de notificaciones en la parte superior.
Evidencia (Figma)	F03
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

3.4.21 RF-021 · Visualizar turno activo

Campo	Detalle
Código	RF-021
Nombre	Visualizar turno activo
Descripción	El sistema debe mostrar el turno de trabajo activo del usuario.
Actor	Administrador, Trabajador
Prioridad (propuesta)	Baja

Campo	Detalle
Criterios de aceptación	1. El menú lateral muestra el turno activo con horario y caja (por ejemplo, "07:00–15:00 · Caja 01"). 2. La variante del panel muestra además el estado "Abierto" y la caja inicial.
Evidencia (Figma)	F02, F03, F04
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

3.5 Reglas de cobro

El modelo de cobro de PARKINLOCK es el definido en el prototipo de Figma: cobro por tiempo con tarifas por tipo de vehículo, cargo por reserva y mensualidades, con pago en efectivo. Figma muestra los valores de cada tarifa, pero no detalla cómo se combinan; las reglas marcadas como propuestas completan esa definición y deben validarse con el equipo.

Código	Regla	Descripción	Evidencia en Figma	Estado
RC-01	Horas completas	Se calculan los minutos entre la entrada y la salida. Cada hora completa se cobra con la tarifa Hora del tipo de vehículo.	F05, F07	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN
RC-02	Minutos restantes	Si sobran de 1 a 30 minutos se cobra una Fracción ; si sobran de 31 a 59 minutos se cobra una Hora más. Si no sobran minutos no se cobra fracción.	F07 ("Fracción · hasta 30 minutos"); comprobado con los 4 cobros de F05 y F06	PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN
RC-03	Tope de día	En una estadía de hasta 12 horas se cobra el menor valor entre el cálculo por horas y la tarifa Día . Si supera 12 horas, se cobra una tarifa Día por cada bloque de 12 horas y el tiempo restante con RC-01 y RC-02, sin superar otra tarifa Día.	F07 ("Día · máximo 12 horas")	PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN
RC-04	Tope de noche	Si la entrada y la salida ocurren dentro de la misma franja de 18:00 a 06:00, se cobra el menor valor entre el cálculo por horas y la tarifa Noche . Si aplican RC-03 y RC-04, se cobra el menor valor.	F07 ("Noche · 18:00–06:00")	PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN
RC-05	Vehículo con mensualidad	Si la placa tiene una mensualidad ACTIVA en la fecha de salida, el valor de la estadía es \$0 y no se registra pago.	F06 (mensualidades por placa)	PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN
RC-06	Cargo por reserva	Al confirmar una reserva se cobra la tarifa Reserva del tipo de vehículo. El pago queda asociado a la reserva y es independiente del cobro por tiempo de la estadía.	F06, F07 ("Reserva · cargo por reserva")	PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN
RC-07	Pago y vigencia de mensualidad	Al crear una mensualidad se cobra la tarifa Mensualidad vigente y el pago queda asociado al plan. La vigencia es de un mes desde la fecha de inicio: termina el día anterior al mismo día del mes siguiente o, si ese día no existe, el último día del mes siguiente (01 oct → 31 oct; 15 oct → 14 nov; 31 ene → 28 feb). Ambas fechas están incluidas; el plan queda VENCIDA al día siguiente de su fecha de fin.	F06 (vigencias "01–31 oct" y "01–30 sep"), F07	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN
RC-08	Pago en efectivo y cambio	Todo pago es en efectivo. Cambio = efectivo recibido – valor. Si el efectivo no alcanza, se muestra el faltante y no se registra la operación.	F05, F11	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN
RC-09	Tarifa aplicada	Se usa la tarifa vigente en la fecha del cobro (la de fecha "vigente desde" más reciente que no sea posterior al día del cobro).	F07 ("Valores usados para calcular cobros desde el 01 oct 2026")	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

Código	Regla	Descripción	Evidencia en Figma	Estado
RC-10	Mensualidad por vencer	Una mensualidad ACTIVA está por vencer cuando faltan 5 días o menos para su fecha de fin (incluido el último día).	F06 ("1 por vencer"; Figma no indica el número de días)	PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN

3.5.1 Verificación con los cobros mostrados en Figma

Placa	Estadía	Cálculo con las reglas	Resultado	Valor en Figma
ABC123 (carro)	08:14 – 10:42 · 2 h 28 min	$2 \times \$5.000 + \text{Fracción } \$5.000 (28 \text{ min})$	\$15.000	\$15.000
RNT209	08:31 – 10:31 · 2 h	$2 \times \$5.000$	\$10.000	\$10.000
GQN54A	07:52 – 09:43 · 1 h 51 min	$1 \times \$3.000 + \text{Hora } \$3.000 (51 \text{ min})$ — coincide con la tarifa de moto	\$6.000	\$6.000
KDM670	18:10 – 22:04 · 3 h 54 min	$3 \times \$5.000 + \text{Hora } \$5.000 (54 \text{ min})$; no supera el tope Noche de \$25.000	\$20.000	\$20.000

Vigencia de las mensualidades: la pantalla F07 rotula la tarifa como "30 días calendario", pero los ejemplos de F06 ("01–31 oct" y "01–30 sep") corresponden a un mes completo. Se adopta la vigencia de un mes (RC-07); el rótulo de F07 debe actualizarse en Figma a "1 mes".

Las tarifas Hora y Fracción tienen hoy el mismo valor, por eso la fracción de 28 minutos de ABC123 cuesta lo mismo que una hora ("3 horas cobradas" en F05). Si el Administrador cambia la tarifa Fracción, RC-02 mantiene el cálculo correcto.

3.6 Matriz de permisos por rol (según Figma)

Área del sistema	Administrador	Trabajador	Requisitos
Panel y mapa	Ver y editar	Ver	RF-005, RF-006, RF-007
Entradas y salidas	Gestionar	Gestionar	RF-008 a RF-012
Reservas	Gestionar	Crear y ver	RF-013
Mensualidades	Gestionar	Solo ver	RF-014
Historial	Ver y exportar	Ver turno propio	RF-015, RF-016
Tarifas y usuarios	Configurar	Sin acceso	RF-017, RF-018

4. Requisitos no funcionales

Los requisitos no funcionales describen las cualidades que debe tener PARKINLOCK. Cada uno indica si está **existente en el código**, **diseñado en Figma** o **propuesto**; ninguno se presenta como implementado si no lo está.

4.1 Resumen por estado

Estado	Cantidad	Requisitos
EXISTENTE EN CÓDIGO	5	RNF-005, RNF-011, RNF-015, RNF-016, RNF-017
DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN	7	RNF-001, RNF-002, RNF-003, RNF-004, RNF-013, RNF-018, RNF-020
PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN	9	RNF-006, RNF-007, RNF-008, RNF-009, RNF-010, RNF-012, RNF-014, RNF-019, RNF-021

4.2 Tabla resumen

Código	Nombre	Categoría	Prioridad	Estado
RNF-001	Diseño adaptable a escritorio y móvil	Usabilidad	Alta	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN
RNF-002	Flujo operativo guiado	Usabilidad	Media	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN
RNF-003	Mensajes de validación y confirmación	Usabilidad	Alta	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN
RNF-004	Idioma y formato local	Usabilidad	Media	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN
RNF-005	Aplicación instalable (PWA)	Compatibilidad	Media	EXISTENTE EN CÓDIGO
RNF-006	Compatibilidad con navegadores	Compatibilidad	Media	PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN
RNF-007	Contraseñas cifradas	Seguridad	Alta	PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN
RNF-008	Autorización por rol en el servidor	Seguridad	Alta	PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN
RNF-009	Protección de credenciales	Seguridad	Alta	PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN

Código	Nombre	Categoría	Prioridad	Estado
RNF-010	Consultas parametrizadas	Seguridad	Alta	PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN
RNF-011	Caché de recursos	Rendimiento	Media	EXISTENTE EN CÓDIGO
RNF-012	Tiempo de respuesta operativo	Rendimiento	Alta	PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN
RNF-013	Actualización del mapa de espacios	Rendimiento	Media	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN
RNF-014	Disponibilidad durante la operación	Disponibilidad	Alta	PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN
RNF-015	Actualización automática de la aplicación	Disponibilidad	Baja	EXISTENTE EN CÓDIGO
RNF-016	Control de versiones	Mantenibilidad	Alta	EXISTENTE EN CÓDIGO
RNF-017	Análisis estático del código	Mantenibilidad	Media	EXISTENTE EN CÓDIGO
RNF-018	Tarifas parametrizables	Escalabilidad	Alta	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN
RNF-019	Crecimiento del parqueadero	Escalabilidad	Media	PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN
RNF-020	Estados identificables sin depender solo del color	Accesibilidad	Media	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN
RNF-021	Contraste y legibilidad	Accesibilidad	Media	PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN

Las prioridades son una propuesta del equipo de documentación.

4.3 Detalle de requisitos no funcionales

4.3.1 RNF-001 · Diseño adaptable a escritorio y móvil

Campo	Detalle
Código	RNF-001
Nombre	Diseño adaptable a escritorio y móvil
Descripción	La interfaz debe adaptarse a pantallas de computador y de teléfono móvil.
Categoría	Usabilidad

Campo	Detalle
Prioridad (propuesta)	Alta
Criterio de aceptación	Las pantallas de inicio de sesión, panel, entrada y salida se visualizan correctamente en escritorio y en móvil, con barra de navegación inferior en móvil.
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN
Respaldo	Pantallas F08 a F11.

4.3.2 RNF-002 · Flujo operativo guiado

Campo	Detalle
Código	RNF-002
Nombre	Flujo operativo guiado
Descripción	El proceso de atención debe mostrarse como una secuencia de pasos visible para el operador.
Categoría	Usabilidad
Prioridad (propuesta)	Media
Criterio de aceptación	Se muestra la secuencia de 7 pasos (Entrada, Placa y espacio, Calcular tiempo, Cobrar, Cambio, Registrar salida, Liberar espacio) indicando el paso actual.
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN
Respaldo	Pantallas F04, F05, F10.

4.3.3 RNF-003 · Mensajes de validación y confirmación

Campo	Detalle
Código	RNF-003
Nombre	Mensajes de validación y confirmación
Descripción	El sistema debe informar el resultado de las validaciones antes de confirmar una operación.
Categoría	Usabilidad
Prioridad (propuesta)	Alta
Criterio de aceptación	Se muestran mensajes como "C-12 está disponible", "Monto suficiente", el valor faltante cuando el pago no alcanza y el resumen previo a registrar la salida.
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN
Respaldo	Pantallas F04, F05, F11.

4.3.4 RNF-004 · Idioma y formato local

Campo	Detalle
Código	RNF-004
Nombre	Idioma y formato local
Descripción	La interfaz debe estar en español y mostrar valores en pesos y horas en formato de 24 horas.

Campo	Detalle
Categoría	Usabilidad
Prioridad (propuesta)	Media
Criterio de aceptación	Todos los textos están en español; los valores se muestran como "\$15.000" y las horas como "08:14".
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN
Respaldo	Todas las pantallas.

4.3.5 RNF-005 · Aplicación instalable (PWA)

Campo	Detalle
Código	RNF-005
Nombre	Aplicación instalable (PWA)
Descripción	La aplicación debe poder instalarse en el dispositivo con nombre e íconos propios.
Categoría	Compatibilidad
Prioridad (propuesta)	Media
Criterio de aceptación	El build genera manifest.webmanifest con nombre, colores, modo standalone e íconos de 64, 192 y 512 px y maskable.
Estado	EXISTENTE EN CÓDIGO
Respaldo	vite.config.js (vite-plugin-pwa), carpeta public/.

4.3.6 RNF-006 · Compatibilidad con navegadores

Campo	Detalle
Código	RNF-006
Nombre	Compatibilidad con navegadores
Descripción	La aplicación debe funcionar en las versiones actuales de Chrome, Edge y Firefox.
Categoría	Compatibilidad
Prioridad (propuesta)	Media
Criterio de aceptación	Las funciones principales se ejecutan sin errores en los tres navegadores.
Estado	PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN
Respaldo	—

4.3.7 RNF-007 · Contraseñas cifradas

Campo	Detalle
Código	RNF-007
Nombre	Contraseñas cifradas
Descripción	Las contraseñas deben almacenarse con un algoritmo de hash seguro, nunca en texto plano.

Campo	Detalle
Categoría	Seguridad
Prioridad (propuesta)	Alta
Criterio de aceptación	La tabla de usuarios solo almacena el hash de la contraseña.
Estado	PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN
Respaldo	Modelo propuesto (documento 06).

4.3.8 RNF-008 · Autorización por rol en el servidor

Campo	Detalle
Código	RNF-008
Nombre	Autorización por rol en el servidor
Descripción	Los permisos de la matriz de roles deben validarse también en el backend, no solo ocultando opciones en la interfaz.
Categoría	Seguridad
Prioridad (propuesta)	Alta
Criterio de aceptación	Una petición de un Trabajador a una función de tarifas o usuarios es rechazada.
Estado	PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN
Respaldo	Matriz de permisos diseñada en F07.

4.3.9 RNF-009 · Protección de credenciales

Campo	Detalle
Código	RNF-009
Nombre	Protección de credenciales
Descripción	Las credenciales de la base de datos deben manejarse en variables de entorno fuera del repositorio.
Categoría	Seguridad
Prioridad (propuesta)	Alta
Criterio de aceptación	Los archivos .env no se suben a Git.
Estado	PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN
Respaldo	El .gitignore del repositorio ya excluye .env y .env.* (existente).

4.3.10 RNF-010 · Consultas parametrizadas

Campo	Detalle
Código	RNF-010
Nombre	Consultas parametrizadas
Descripción	Las consultas a la base de datos deben usar parámetros para prevenir inyección SQL.

Campo	Detalle
Categoría	Seguridad
Prioridad (propuesta)	Alta
Criterio de aceptación	Ninguna consulta concatena datos digitados por el usuario.
Estado	PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN
Respaldo	Consultas del documento 06.

4.3.11 RNF-011 · Caché de recursos

Campo	Detalle
Código	RNF-011
Nombre	Caché de recursos
Descripción	Los recursos estáticos deben almacenarse en caché mediante service worker para acelerar las cargas.
Categoría	Rendimiento
Prioridad (propuesta)	Media
Criterio de aceptación	El build genera sw.js (Workbox) con la lista de recursos en caché.
Estado	EXISTENTE EN CÓDIGO
Respaldo	vite.config.js, salida de npm run build.

4.3.12 RNF-012 · Tiempo de respuesta operativo

Campo	Detalle
Código	RNF-012
Nombre	Tiempo de respuesta operativo
Descripción	Registrar una entrada, buscar una placa o registrar una salida debe responder en menos de 2 segundos en condiciones normales.
Categoría	Rendimiento
Prioridad (propuesta)	Alta
Criterio de aceptación	Medición del tiempo de respuesta de las operaciones RF-008, RF-009 y RF-012.
Estado	PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN
Respaldo	—

4.3.13 RNF-013 · Actualización del mapa de espacios

Campo	Detalle
Código	RNF-013
Nombre	Actualización del mapa de espacios
Descripción	El mapa y los indicadores deben reflejar cada entrada, salida o reserva sin recargar la página.

Campo	Detalle
Categoría	Rendimiento
Prioridad (propuesta)	Media
Criterio de aceptación	Tras registrar una entrada, el espacio aparece como ocupado en el panel.
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN
Respaldo	Textos "tiempo real" y "En vivo" en F02 y F03.

4.3.14 RNF-014 · Disponibilidad durante la operación

Campo	Detalle
Código	RNF-014
Nombre	Disponibilidad durante la operación
Descripción	El sistema debe estar disponible durante los turnos de atención del parqueadero.
Categoría	Disponibilidad
Prioridad (propuesta)	Alta
Criterio de aceptación	Horario de disponibilidad acordado con la institución: [Pendiente de completar] .
Estado	PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN
Respaldo	—

4.3.15 RNF-015 · Actualización automática de la aplicación

Campo	Detalle
Código	RNF-015
Nombre	Actualización automática de la aplicación
Descripción	Cuando se publique una nueva versión, la aplicación instalada debe actualizarse automáticamente.
Categoría	Disponibilidad
Prioridad (propuesta)	Baja
Criterio de aceptación	El service worker está configurado con registerType "autoUpdate".
Estado	EXISTENTE EN CÓDIGO
Respaldo	vite.config.js.

4.3.16 RNF-016 · Control de versiones

Campo	Detalle
Código	RNF-016
Nombre	Control de versiones
Descripción	El código debe versionarse en GitHub separando frontend y backend.

Campo	Detalle
Categoría	Mantenibilidad
Prioridad (propuesta)	Alta
Criterio de aceptación	El repositorio contiene la carpeta frontend-Parkinlock- y el historial de cambios.
Estado	EXISTENTE EN CÓDIGO
Respaldo	Repositorio GitHub (la carpeta del backend aún no está registrada porque está vacía).

4.3.17 RNF-017 · Análisis estático del código

Campo	Detalle
Código	RNF-017
Nombre	Análisis estático del código
Descripción	El código del frontend debe revisarse con un linter.
Categoría	Mantenibilidad
Prioridad (propuesta)	Media
Criterio de aceptación	El comando npm run lint ejecuta oxlint con reglas de React.
Estado	EXISTENTE EN CÓDIGO
Respaldo	package.json, .oxlintc.json.

4.3.18 RNF-018 · Tarifas parametrizables

Campo	Detalle
Código	RNF-018
Nombre	Tarifas parametrizables
Descripción	Los valores de cobro deben poder cambiarse desde la configuración, sin modificar el código.
Categoría	Escalabilidad
Prioridad (propuesta)	Alta
Criterio de aceptación	El Administrador cambia una tarifa, pulsa "Guardar cambios" y el nuevo valor se usa en los siguientes cobros.
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN
Respaldo	Pantalla F07.

4.3.19 RNF-019 · Crecimiento del parqueadero

Campo	Detalle
Código	RNF-019
Nombre	Crecimiento del parqueadero

Campo	Detalle
Descripción	El modelo de datos debe permitir agregar espacios y tipos de vehículo sin cambiar la estructura de las tablas.
Categoría	Escalabilidad
Prioridad (propuesta)	Media
Criterio de aceptación	Agregar un espacio nuevo solo requiere insertar un registro.
Estado	PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN
Respaldo	Modelo propuesto (documentos 05 y 06).

4.3.20 RNF-020 · Estados identificables sin depender solo del color

Campo	Detalle
Código	RNF-020
Nombre	Estados identificables sin depender solo del color
Descripción	Los estados de los espacios y registros deben indicarse con texto además del color.
Categoría	Accesibilidad
Prioridad (propuesta)	Media
Criterio de aceptación	Cada espacio muestra el texto "Libre", "Ocupado" o "Reserva" junto al color correspondiente.
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN
Respaldo	Pantallas F02, F03, F09.

4.3.21 RNF-021 · Contraste y legibilidad

Campo	Detalle
Código	RNF-021
Nombre	Contraste y legibilidad
Descripción	Los textos deben tener contraste suficiente y tamaño legible.
Categoría	Accesibilidad
Prioridad (propuesta)	Media
Criterio de aceptación	Los textos cumplen la relación de contraste recomendada por WCAG 2.1 nivel AA.
Estado	PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN
Respaldo	—

5. Historias de usuario

Cada historia de usuario corresponde a un requisito funcional (relación 1 a 1). Los actores son los definidos en Figma: **Administrador** y **Trabajador**. Todas las historias describen funcionalidades **diseñadas en Figma y pendientes de implementación**.

5.1 Tabla resumen

Código	Actor	Historia	RF	Prioridad
HU-001	Administrador o Trabajador	Como administrador o trabajador, quiero iniciar sesión seleccionando mi rol e ingresando mi usuario y contraseña, para acceder a las funciones que me corresponden.	RF-001	Alta
HU-002	Administrador o Trabajador	Como administrador o trabajador, quiero tener una opción para recuperar mi clave, para volver a ingresar si la olvido.	RF-002	Baja
HU-003	Administrador o Trabajador	Como administrador o trabajador, quiero cerrar mi sesión desde el menú, para evitar que otra persona use mi cuenta.	RF-003	Media
HU-004	Administrador	Como administrador, quiero que cada rol solo vea y use las funciones permitidas, para proteger la configuración y la información sensible.	RF-004	Alta
HU-005	Administrador o Trabajador	Como administrador o trabajador, quiero ver los indicadores del día en el panel, para conocer rápidamente el estado de la operación.	RF-005	Media
HU-006	Administrador o Trabajador	Como administrador o trabajador, quiero ver el mapa de espacios con su estado, para saber qué espacios están libres, ocupados o reservados.	RF-006	Alta
HU-007	Administrador o Trabajador	Como administrador o trabajador, quiero consultar los últimos movimientos, para hacer seguimiento a la actividad reciente.	RF-007	Media
HU-008	Trabajador	Como trabajador, quiero registrar la entrada de un vehículo con su placa y espacio, para dejar constancia del ingreso y ocupar el espacio.	RF-008	Alta
HU-009	Trabajador	Como trabajador, quiero buscar un vehículo por su placa, para iniciar rápidamente el proceso de salida.	RF-009	Alta
HU-010	Trabajador	Como trabajador, quiero que el sistema calcule el tiempo y el valor a cobrar, para cobrar de forma correcta según las tarifas.	RF-010	Alta
HU-011	Trabajador	Como trabajador, quiero registrar el efectivo recibido y ver el cambio, para entregar el cambio correcto y detectar si falta dinero.	RF-011	Alta
HU-012	Trabajador	Como trabajador, quiero registrar la salida y liberar el espacio, para que el espacio quede disponible para otro vehículo.	RF-012	Alta

Código	Actor	Historia	RF	Prioridad
HU-013	Administrador o Trabajador	Como administrador o trabajador, quiero consultar las reservas del día y crear nuevas reservas, para garantizar espacios a los clientes que reservan.	RF-013	Media
HU-014	Administrador	Como administrador, quiero consultar y crear mensualidades, para controlar los planes vigentes y vencidos.	RF-014	Media
HU-015	Administrador o Trabajador	Como administrador o trabajador, quiero consultar el historial de movimientos, para revisar entradas, salidas y pagos realizados.	RF-015	Media
HU-016	Administrador	Como administrador, quiero exportar el historial, para analizar o respaldar la información fuera del sistema.	RF-016	Baja
HU-017	Administrador	Como administrador, quiero configurar las tarifas por tipo de vehículo, para que los cobros se calculen con los valores vigentes.	RF-017	Alta
HU-018	Administrador	Como administrador, quiero gestionar los usuarios del sistema, para controlar quién puede operar el parqueadero.	RF-018	Baja
HU-019	Administrador o Trabajador	Como administrador o trabajador, quiero consultar los vehículos en operación, para saber qué vehículos están dentro y cuánto tiempo llevan.	RF-019	Media
HU-020	Administrador o Trabajador	Como administrador o trabajador, quiero ver alertas operativas, para atender a tiempo reservas por vencer y fallas.	RF-020	Baja
HU-021	Administrador o Trabajador	Como administrador o trabajador, quiero ver mi turno activo, para saber en qué turno y caja estoy operando.	RF-021	Baja

Las prioridades se heredan del requisito funcional relacionado y son una propuesta.

5.2 Detalle de historias de usuario

5.2.1 HU-001 · Iniciar sesión con selección de rol

Campo	Detalle
Código	HU-001
Actor	Administrador o Trabajador
Historia	Como administrador o trabajador, quiero iniciar sesión seleccionando mi rol e ingresando mi usuario y contraseña, para acceder a las funciones que me corresponden.
Requisito relacionado	RF-001

Campo	Detalle
Criterios de aceptación	1. El formulario muestra el selector de rol con las opciones "Administrador" y "Trabajador". 2. Solicita "Correo o usuario" y "Contraseña". 3. La contraseña se muestra oculta y puede hacerse visible con el ícono de ojo. 4. Incluye la opción "Recordar acceso". 5. Al pulsar "Entrar al sistema" (escritorio) o "Entrar" (móvil) se accede al panel del sistema.
Prioridad (propuesta)	Alta
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

5.2.2 HU-002 · Recuperar contraseña

Campo	Detalle
Código	HU-002
Actor	Administrador o Trabajador
Historia	Como administrador o trabajador, quiero tener una opción para recuperar mi clave, para volver a ingresar si la olvido.
Requisito relacionado	RF-002
Criterios de aceptación	1. La pantalla de inicio de sesión muestra el enlace "¿Olvidaste tu clave?". 2. Se muestra el mensaje "¿Necesitas ayuda? Contacta al administrador de la sede." 3. Pendiente: el prototipo no incluye la pantalla ni el flujo de recuperación.
Prioridad (propuesta)	Baja
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

5.2.3 HU-003 · Cerrar sesión

Campo	Detalle
Código	HU-003
Actor	Administrador o Trabajador
Historia	Como administrador o trabajador, quiero cerrar mi sesión desde el menú, para evitar que otra persona use mi cuenta.
Requisito relacionado	RF-003
Criterios de aceptación	1. El menú lateral muestra el nombre y rol del usuario autenticado (por ejemplo, "Andrea Mora · Administrador", "Carlos Silva · Trabajador"). 2. Junto al usuario se muestra un ícono de salida para cerrar la sesión. 3. Pendiente: el prototipo no muestra la pantalla de confirmación ni el destino tras cerrar sesión.
Prioridad (propuesta)	Media
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

5.2.4 HU-004 · Control de acceso por rol

Campo	Detalle
Código	HU-004
Actor	Administrador
Historia	Como administrador, quiero que cada rol solo vea y use las funciones permitidas, para proteger la configuración y la información sensible.
Requisito relacionado	RF-004
Criterios de aceptación	1. El Administrador ve en el menú la opción "Configuración"; el Trabajador no la ve. 2. Panel y mapa: Administrador "Ver y editar"; Trabajador "Ver". 3. Entradas y salidas: ambos roles "Gestionar". 4. Reservas: Administrador "Gestionar"; Trabajador "Crear y ver". 5. Mensualidades: Administrador "Gestionar"; Trabajador "Solo ver". 6. Historial: Administrador "Ver y exportar"; Trabajador "Ver turno propio". 7. Tarifas y usuarios: Administrador "Configurar"; Trabajador "Sin acceso".
Prioridad (propuesta)	Alta
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

5.2.5 HU-005 · Visualizar panel general de indicadores

Campo	Detalle
Código	HU-005
Actor	Administrador o Trabajador
Historia	Como administrador o trabajador, quiero ver los indicadores del día en el panel, para conocer rápidamente el estado de la operación.
Requisito relacionado	RF-005
Criterios de aceptación	1. Muestra vehículos dentro (con desglose de carros y motos), espacios disponibles, ocupados y reservados, y el valor recaudado del día. 2. Muestra el porcentaje de ocupación actual y el pico esperado. 3. Ofrece accesos directos: "Registrar entrada", "Cobrar salida" y "Nueva reserva" (escritorio); "Entrada" y "Cobrar" (móvil). 4. Muestra la fecha y la hora actual.
Prioridad (propuesta)	Media
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

5.2.6 HU-006 · Visualizar mapa de espacios

Campo	Detalle
Código	HU-006
Actor	Administrador o Trabajador

Campo	Detalle
Historia	Como administrador o trabajador, quiero ver el mapa de espacios con su estado, para saber qué espacios están libres, ocupados o reservados.
Requisito relacionado	RF-006
Criterios de aceptación	1. Cada espacio muestra su código (C- para carros, M- para motos) y su estado: Libre/Disponible, Ocupado o Reserva. 2. Los espacios ocupados muestran la placa del vehículo. 3. Se muestra una leyenda con el total de espacios disponibles, ocupados y reservados. 4. En móvil se muestra un "Mapa rápido" con la opción "Ver todos".
Prioridad (propuesta)	Alta
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

5.2.7 HU-007 · Consultar últimos movimientos

Campo	Detalle
Código	HU-007
Actor	Administrador o Trabajador
Historia	Como administrador o trabajador, quiero consultar los últimos movimientos, para hacer seguimiento a la actividad reciente.
Requisito relacionado	RF-007
Criterios de aceptación	1. Cada movimiento muestra hora, placa, tipo de movimiento (entrada, salida con valor, en parqueadero), espacio y trabajador. 2. Incluye el enlace "Ver historial".
Prioridad (propuesta)	Media
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

5.2.8 HU-008 · Registrar entrada de vehículo

Campo	Detalle
Código	HU-008
Actor	Trabajador
Historia	Como trabajador, quiero registrar la entrada de un vehículo con su placa y espacio, para dejar constancia del ingreso y ocupar el espacio.
Requisito relacionado	RF-008

Campo	Detalle
Criterios de aceptación	1. Permite seleccionar el tipo de vehículo: Carro o Moto. 2. Solicita la placa y valida su formato ("Formato validado · sin coincidencias activas"). 3. Permite seleccionar el espacio de una lista filtrada por tipo de vehículo ("Espacio para carro"). 4. La hora de entrada se registra automáticamente al confirmar. 5. Indica si el espacio está disponible ("C-12 está disponible · Se marcará como ocupado al guardar"). 6. Muestra los espacios cercanos disponibles y una tarjeta con placa, espacio, hora de entrada y tipo. 7. Botones "Cancelar" y "Guardar placa y espacio"; al guardar, el espacio pasa a ocupado.
Prioridad (propuesta)	Alta
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

5.2.9 HU-009 · Buscar vehículo por placa para la salida

Campo	Detalle
Código	HU-009
Actor	Trabajador
Historia	Como trabajador, quiero buscar un vehículo por su placa, para iniciar rápidamente el proceso de salida.
Requisito relacionado	RF-009
Criterios de aceptación	1. Incluye el campo "Buscar vehículo por placa" y el botón "Buscar". 2. Si la placa existe, muestra el estado "Encontrado", el tipo de vehículo, la tarifa y el espacio ocupado. 3. Muestra la hora de entrada registrada, la hora de salida calculada y el tiempo utilizado.
Prioridad (propuesta)	Alta
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

5.2.10 HU-010 · Calcular tiempo y valor a cobrar

Campo	Detalle
Código	HU-010
Actor	Trabajador
Historia	Como trabajador, quiero que el sistema calcule el tiempo y el valor a cobrar, para cobrar de forma correcta según las tarifas.
Requisito relacionado	RF-010

Campo	Detalle
Criterios de aceptación	1. Calcula el tiempo utilizado entre la entrada y la salida (por ejemplo, "2 h 28 min"). 2. Muestra el desglose del cobro por hora y por fracción iniciada. 3. Muestra el "Total a pagar". 4. Usa las tarifas configuradas para el tipo de vehículo (ver RF-017). 5. Aplica las reglas de cobro RC-01 a RC-05 (tabla "Reglas de cobro").
Prioridad (propuesta)	Alta
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

5.2.11 HU-011 · Recibir pago y calcular cambio

Campo	Detalle
Código	HU-011
Actor	Trabajador
Historia	Como trabajador, quiero registrar el efectivo recibido y ver el cambio, para entregar el cambio correcto y detectar si falta dinero.
Requisito relacionado	RF-011
Criterios de aceptación	1. Muestra el medio de pago "Efectivo" y el total. 2. Solicita el "Efectivo recibido". 3. Si el monto es suficiente, muestra "Monto suficiente" y el "Cambio para el cliente". 4. Si el monto no alcanza, muestra un aviso con el valor faltante (por ejemplo, "Con \$12.000, faltan \$3.000 para completar"). 5. El mismo registro de pago se usa para salidas, cargos de reserva y mensualidades (RC-06 a RC-08).
Prioridad (propuesta)	Alta
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

5.2.12 HU-012 · Registrar salida y liberar espacio

Campo	Detalle
Código	HU-012
Actor	Trabajador
Historia	Como trabajador, quiero registrar la salida y liberar el espacio, para que el espacio quede disponible para otro vehículo.
Requisito relacionado	RF-012

Campo	Detalle
Criterios de aceptación	1. Antes de confirmar, muestra un resumen con hora de salida, pago, cambio y el cambio de estado del espacio (Ocupado → Disponible). 2. El botón "Registrar salida y liberar [espacio]" registra la salida. 3. Al confirmar, el espacio cambia de ocupado a disponible. 4. El proceso sigue la secuencia: Entrada, Placa y espacio, Calcular tiempo, Cobrar, Cambio, Registrar salida, Liberar espacio.
Prioridad (propuesta)	Alta
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

5.2.13 HU-013 · Gestionar reservas

Campo	Detalle
Código	HU-013
Actor	Administrador o Trabajador
Historia	Como administrador o trabajador, quiero consultar las reservas del día y crear nuevas reservas, para garantizar espacios a los clientes que reservan.
Requisito relacionado	RF-013
Criterios de aceptación	1. Muestra las reservas de hoy con placa, cliente, horario, espacio asignado y estado (Confirmada o Pendiente). 2. Muestra el resumen de reservas programadas y espacios aún sin asignar. 3. Incluye el botón "Nueva reserva" (también disponible en el panel). 4. El panel móvil muestra la reserva próxima. 5. Al confirmar la reserva se cobra el cargo por reserva del tipo de vehículo (RC-06, regla propuesta). 6. Pendiente: el prototipo no incluye el formulario de nueva reserva.
Prioridad (propuesta)	Media
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

5.2.14 HU-014 · Gestionar mensualidades

Campo	Detalle
Código	HU-014
Actor	Administrador
Historia	Como administrador, quiero consultar y crear mensualidades, para controlar los planes vigentes y vencidos.
Requisito relacionado	RF-014

Campo	Detalle
Criterios de aceptación	1. Muestra la lista de mensualidades con placa, titular, plan (carro o moto mensual), vigencia, valor y estado (Activa o Vencida). 2. Muestra el resumen de planes activos, por vencer (5 días o menos para la fecha de fin, RC-10) y vencidos. 3. La vigencia de cada plan es de un mes desde la fecha de inicio, ambas fechas incluidas (por ejemplo, "01-31 oct" o "01-30 sep") (RC-07). 4. Incluye el botón "Nueva mensualidad". 5. El valor del plan es la tarifa de mensualidad vigente y su pago queda registrado (RC-07). 6. Un vehículo con mensualidad activa no paga por tiempo al salir (RC-05, regla propuesta). 7. Pendiente: el prototipo no incluye el formulario de nueva mensualidad.
Prioridad (propuesta)	Media
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

5.2.15 HU-015 · Consultar historial de movimientos

Campo	Detalle
Código	HU-015
Actor	Administrador o Trabajador
Historia	Como administrador o trabajador, quiero consultar el historial de movimientos, para revisar entradas, salidas y pagos realizados.
Requisito relacionado	RF-015
Criterios de aceptación	1. Muestra placa, fecha, hora de entrada, hora de salida, tiempo, valor pagado y trabajador. 2. Permite buscar por placa. 3. Permite filtrar por fecha.
Prioridad (propuesta)	Media
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

5.2.16 HU-016 · Exportar historial

Campo	Detalle
Código	HU-016
Actor	Administrador
Historia	Como administrador, quiero exportar el historial, para analizar o respaldar la información fuera del sistema.
Requisito relacionado	RF-016
Criterios de aceptación	1. El historial incluye el botón "Exportar". 2. Solo el Administrador tiene el permiso "Ver y exportar". 3. Pendiente: el prototipo no define el formato del archivo exportado.

Campo	Detalle
Prioridad (propuesta)	Baja
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

5.2.17 HU-017 · Configurar tarifas

Campo	Detalle
Código	HU-017
Actor	Administrador
Historia	Como administrador, quiero configurar las tarifas por tipo de vehículo, para que los cobros se calculen con los valores vigentes.
Requisito relacionado	RF-017
Criterios de aceptación	1. Permite editar el valor para Carro y Moto de los conceptos: Hora (cada hora iniciada), Fracción (hasta 30 minutos), Noche (18:00–06:00), Día (máximo 12 horas), Reserva (cargo por reserva) y Mensualidad (vigencia de un mes, RC-07; la pantalla la rotula como "30 días calendario"). 2. Incluye el botón "Guardar cambios". 3. Muestra un ejemplo del cálculo con las tarifas vigentes. 4. La pantalla indica "Configuración exclusiva del administrador".
Prioridad (propuesta)	Alta
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

5.2.18 HU-018 · Gestionar usuarios

Campo	Detalle
Código	HU-018
Actor	Administrador
Historia	Como administrador, quiero gestionar los usuarios del sistema, para controlar quién puede operar el parqueadero.
Requisito relacionado	RF-018
Criterios de aceptación	1. La pantalla de Configuración incluye el botón "Gestionar usuarios". 2. Solo el Administrador tiene acceso ("Tarifas y usuarios: Configurar"). 3. Pendiente: el prototipo no incluye la pantalla de gestión de usuarios.
Prioridad (propuesta)	Baja
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

5.2.19 HU-019 · Consultar vehículos en operación

Campo	Detalle
Código	HU-019

Campo	Detalle
Actor	Administrador o Trabajador
Historia	Como administrador o trabajador, quiero consultar los vehículos en operación, para saber qué vehículos están dentro y cuánto tiempo llevan.
Requisito relacionado	RF-019
Criterios de aceptación	1. Muestra placa, cliente, tipo, hora de entrada, permanencia, espacio, estado (Estacionado, Por vencer, Reserva) y acciones. 2. Permite buscar por placa o cliente y filtrar. 3. Incluye paginación y la opción "Ver todos".
Prioridad (propuesta)	Media
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

5.2.20 HU-020 · Mostrar alertas operativas

Campo	Detalle
Código	HU-020
Actor	Administrador o Trabajador
Historia	Como administrador o trabajador, quiero ver alertas operativas, para atender a tiempo reservas por vencer y fallas.
Requisito relacionado	RF-020
Criterios de aceptación	1. Muestra el número de alertas activas. 2. Ejemplos visibles: "Reserva A05 vence en 13 min · Cliente aún no registra ingreso" y "Sensor E12 sin respuesta". 3. Incluye un ícono de notificaciones en la parte superior.
Prioridad (propuesta)	Baja
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

5.2.21 HU-021 · Visualizar turno activo

Campo	Detalle
Código	HU-021
Actor	Administrador o Trabajador
Historia	Como administrador o trabajador, quiero ver mi turno activo, para saber en qué turno y caja estoy operando.
Requisito relacionado	RF-021
Criterios de aceptación	1. El menú lateral muestra el turno activo con horario y caja (por ejemplo, "07:00–15:00 · Caja 01"). 2. La variante del panel muestra además el estado "Abierto" y la caja inicial.

Campo	Detalle
Prioridad (propuesta)	Baja
Estado	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN

6. Modelo entidad-relación

Modelo propuesto a partir de las funcionalidades diseñadas en Figma

El repositorio **no contiene una base de datos**: no hay scripts SQL, migraciones, modelos ni conexión configurada. El siguiente modelo es una **propuesta** que incluye únicamente las entidades necesarias para soportar los 21 requisitos funcionales. **No corresponde a una base de datos implementada**.

6.1 Diagrama entidad-relación propuesto

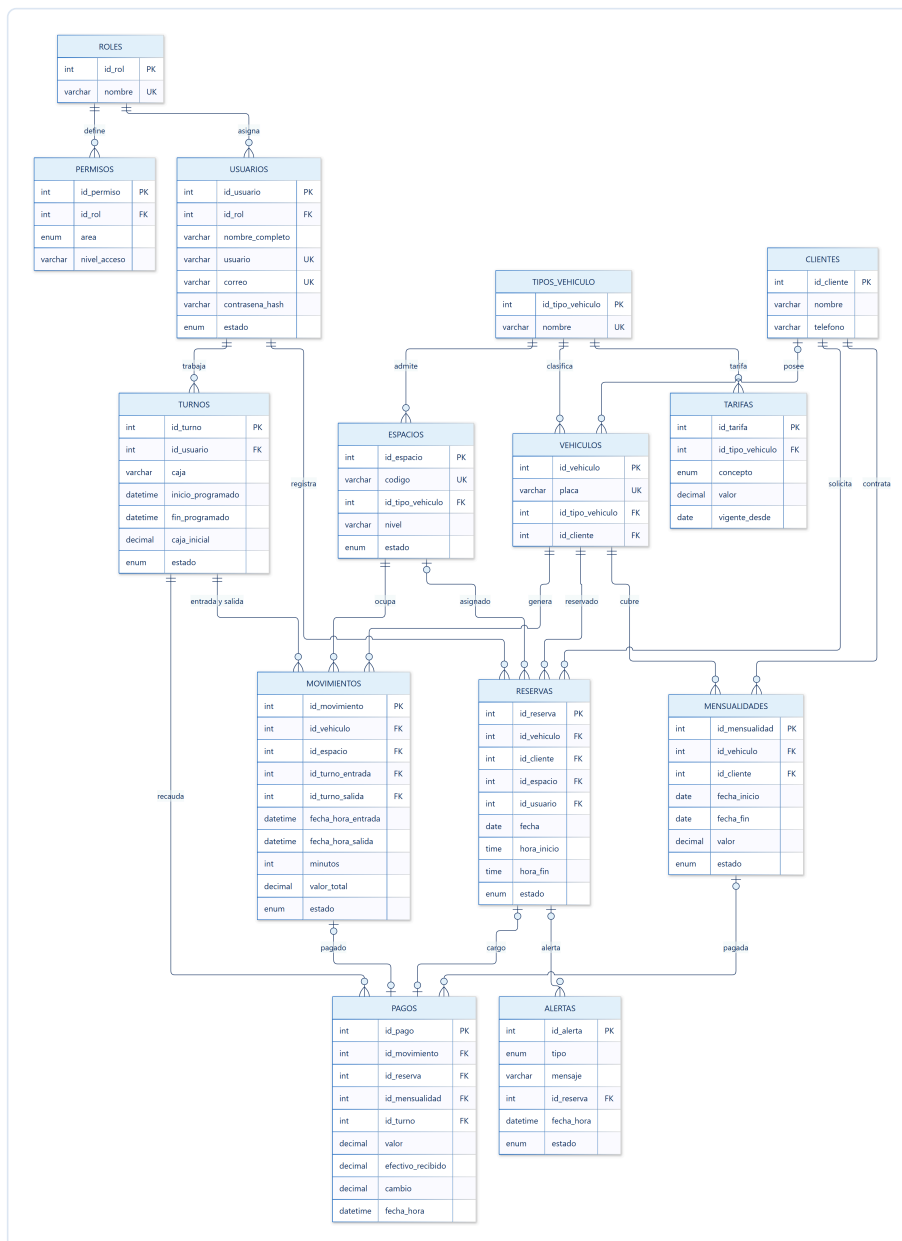


Figura 1. Modelo entidad-relación propuesto de PARKINLOCK (pendiente de implementación).

Notación: PK = llave primaria, FK = llave foránea, UK = valor único. "||" exactamente uno, "lo" cero o uno, "o" cero o muchos.

6.2 Entidades propuestas

Entidad	Propósito	Requisitos que soporta
roles	Roles de acceso: ADMINISTRADOR y TRABAJADOR.	RF-001, RF-004
permisos	Nivel de acceso de cada rol a cada área (matriz de permisos).	RF-004
usuarios	Personas que usan el sistema (Administrador o Trabajador).	RF-001 a RF-004, RF-018
turnos	Turnos de trabajo con caja asignada.	RF-021, RF-015
tipos_vehiculo	Tipos de vehículo: CARRO y MOTO.	RF-006, RF-008, RF-017
espacios	Espacios de estacionamiento y su estado.	RF-006, RF-008, RF-012, RF-013
clientes	Personas o empresas asociadas a vehículos, reservas o mensualidades.	RF-013, RF-014, RF-019
vehiculos	Vehículos identificados por placa.	RF-008, RF-009, RF-019
tarifas	Valores de cobro por concepto y tipo de vehículo.	RF-010, RF-017
movimientos	Estadía de un vehículo: entrada, salida, tiempo y valor.	RF-007 a RF-012, RF-015, RF-019
pagos	Pagos en efectivo con valor recibido y cambio. Cada pago corresponde a una sola operación: una salida, una reserva o una mensualidad.	RF-005, RF-011, RF-013, RF-014
reservas	Reservas de espacio para una placa y un horario.	RF-013, RF-020
mensualidades	Planes mensuales de un vehículo.	RF-014
alertas	Alertas operativas mostradas en el panel.	RF-020

6.3 Atributos por entidad

6.3.1 roles

Roles de acceso: ADMINISTRADOR y TRABAJADOR.

Atributo	Tipo	Llave / restricción
id_rol	INT	PK, AUTO_INCREMENT
nombre	VARCHAR(30)	NOT NULL, UNIQUE
descripcion	VARCHAR(150)	NULL

6.3.2 permisos

Nivel de acceso de cada rol a cada área (matriz de permisos).

Atributo	Tipo	Llave / restricción
id_permiso	INT	PK, AUTO_INCREMENT
id_rol	INT	FK → roles, NOT NULL

Atributo	Tipo	Llave / restricción
area	ENUM('PANEL', 'ENTRADAS_SALIDAS', 'RESERVAS', 'MENSUALIDADES', 'HISTORIAL', 'TARIFAS_USUARIOS')	NOT NULL
nivel_acceso	VARCHAR(30)	NOT NULL
(id_rol, area)	–	UNIQUE

6.3.3 usuarios

Personas que usan el sistema (Administrador o Trabajador).

Atributo	Tipo	Llave / restricción
id_usuario	INT	PK, AUTO_INCREMENT
id_rol	INT	FK → roles, NOT NULL
nombre_completo	VARCHAR(120)	NOT NULL
usuario	VARCHAR(50)	NOT NULL, UNIQUE
correo	VARCHAR(120)	NULL, UNIQUE
contrasena_hash	VARCHAR(255)	NOT NULL
estado	ENUM('ACTIVO', 'INACTIVO')	DEFAULT 'ACTIVO'
fecha_creacion	DATETIME	DEFAULT CURRENT_TIMESTAMP

6.3.4 turnos

Turnos de trabajo con caja asignada.

Atributo	Tipo	Llave / restricción
id_turno	INT	PK, AUTO_INCREMENT
id_usuario	INT	FK → usuarios, NOT NULL
caja	VARCHAR(20)	NOT NULL
inicio_programado	DATETIME	NOT NULL
fin_programado	DATETIME	NOT NULL
caja_inicial	DECIMAL(12,2)	DEFAULT 0
estado	ENUM('ABIERTO', 'CERRADO')	DEFAULT 'ABIERTO'

6.3.5 tipos_vehiculo

Tipos de vehículo: CARRO y MOTO.

Atributo	Tipo	Llave / restricción
id_tipo_vehiculo	INT	PK, AUTO_INCREMENT
nombre	VARCHAR(20)	NOT NULL, UNIQUE

6.3.6 espacios

Espacios de estacionamiento y su estado.

Atributo	Tipo	Llave / restricción
id_espacio	INT	PK, AUTO_INCREMENT
codigo	VARCHAR(10)	NOT NULL, UNIQUE
id_tipo_vehiculo	INT	FK → tipos_vehiculo, NOT NULL
nivel	VARCHAR(20)	NULL
estado	ENUM('DISPONIBLE', 'OCUPADO', 'RESERVADO')	DEFAULT 'DISPONIBLE'

6.3.7 clientes

Personas o empresas asociadas a vehículos, reservas o mensualidades.

Atributo	Tipo	Llave / restricción
id_cliente	INT	PK, AUTO_INCREMENT
nombre	VARCHAR(120)	NOT NULL
telefono	VARCHAR(20)	NULL

6.3.8 vehiculos

Vehículos identificados por placa.

Atributo	Tipo	Llave / restricción
id_vehiculo	INT	PK, AUTO_INCREMENT
placa	VARCHAR(10)	NOT NULL, UNIQUE
id_tipo_vehiculo	INT	FK → tipos_vehiculo, NOT NULL
id_cliente	INT	FK → clientes, NULL

6.3.9 tarifas

Valores de cobro por concepto y tipo de vehículo.

Atributo	Tipo	Llave / restricción
id_tarifa	INT	PK, AUTO_INCREMENT
id_tipo_vehiculo	INT	FK → tipos_vehiculo, NOT NULL
concepto	ENUM('HORA', 'FRACCION', 'NOCHE', 'DIA', 'RESERVA', 'MENSUALIDAD')	NOT NULL
valor	DECIMAL(12,2)	NOT NULL, CHECK ≥ 0
vigente_desde	DATE	NOT NULL
(id_tipo_vehiculo, concepto, vigente_desde)	–	UNIQUE

6.3.10 movimientos

Estadía de un vehículo: entrada, salida, tiempo y valor.

Atributo	Tipo	Llave / restricción
id_movimiento	INT	PK, AUTO_INCREMENT
id_vehiculo	INT	FK → vehiculos, NOT NULL
id_espacio	INT	FK → espacios, NOT NULL
id_turno_entrada	INT	FK → turnos, NOT NULL
id_turno_salida	INT	FK → turnos, NULL
fecha_hora_entrada	DATETIME	NOT NULL
fecha_hora_salida	DATETIME	NULL
minutos	INT	NULL
valor_total	DECIMAL(12,2)	NULL
estado	ENUM('EN_PARQUEADERO', 'FINALIZADO')	DEFAULT 'EN_PARQUEADERO'

6.3.11 pagos

Pagos en efectivo con valor recibido y cambio. Cada pago corresponde a una sola operación: una salida, una reserva o una mensualidad.

Atributo	Tipo	Llave / restricción
id_pago	INT	PK, AUTO_INCREMENT
id_movimiento	INT	FK → movimientos, NULL, UNIQUE
id_reserva	INT	FK → reservas, NULL, UNIQUE
id_mensualidad	INT	FK → mensualidades, NULL
id_turno	INT	FK → turnos, NOT NULL
metodo	ENUM('EFECTIVO')	DEFAULT 'EFECTIVO'
valor	DECIMAL(12,2)	NOT NULL
efectivo_recibido	DECIMAL(12,2)	NOT NULL
cambio	DECIMAL(12,2)	NOT NULL
fecha_hora	DATETIME	DEFAULT CURRENT_TIMESTAMP

6.3.12 reservas

Reservas de espacio para una placa y un horario.

Atributo	Tipo	Llave / restricción
id_reserva	INT	PK, AUTO_INCREMENT
id_vehiculo	INT	FK → vehiculos, NOT NULL

Atributo	Tipo	Llave / restricción
id_cliente	INT	FK → clientes, NOT NULL
id_espacio	INT	FK → espacios, NULL (sin asignar)
id_usuario	INT	FK → usuarios, NOT NULL
fecha	DATE	NOT NULL
hora_inicio	TIME	NOT NULL
hora_fin	TIME	NOT NULL
estado	ENUM('PENDIENTE', 'CONFIRMADA')	DEFAULT 'PENDIENTE'

6.3.13 mensualidades

Planes mensuales de un vehículo.

Atributo	Tipo	Llave / restricción
id_mensualidad	INT	PK, AUTO_INCREMENT
id_vehiculo	INT	FK → vehiculos, NOT NULL
id_cliente	INT	FK → clientes, NOT NULL
fecha_inicio	DATE	NOT NULL
fecha_fin	DATE	NOT NULL
valor	DECIMAL(12,2)	NOT NULL
estado	ENUM('ACTIVA', 'VENCIDA')	DEFAULT 'ACTIVA'

6.3.14 alertas

Alertas operativas mostradas en el panel.

Atributo	Tipo	Llave / restricción
id_alerta	INT	PK, AUTO_INCREMENT
tipo	ENUM('RESERVA_POR_VENCER', 'SENSOR')	NOT NULL
mensaje	VARCHAR(200)	NOT NULL
id_reserva	INT	FK → reservas, NULL
fecha_hora	DATETIME	DEFAULT CURRENT_TIMESTAMP
estado	ENUM('ACTIVA', 'ATENDIDA')	DEFAULT 'ACTIVA'

6.4 Relaciones y cardinalidades

Relación	Cardinalidad	Llave foránea	Descripción
roles — usuarios	1 : N	usuarios.id_rol	Cada usuario tiene un rol; un rol se asigna a muchos usuarios.

Relación	Cardinalidad	Llave foránea	Descripción
roles — permisos	1 : N	permisos.id_rol	Cada rol tiene un nivel de acceso por área.
usuarios — turnos	1 : N	turnos.id_usuario	Un usuario trabaja muchos turnos.
usuarios — reservas	1 : N	reservas.id_usuario	Usuario que creó la reserva.
tipos_vehiculo — espacios	1 : N	espacios.id_tipo_vehiculo	Cada espacio admite un tipo (carro o moto).
tipos_vehiculo — vehiculos	1 : N	vehiculos.id_tipo_vehiculo	Cada vehículo es carro o moto.
tipos_vehiculo — tarifas	1 : N	tarifas.id_tipo_vehiculo	Cada tipo tiene sus tarifas por concepto.
clientes — vehiculos	0..1 : N	vehiculos.id_cliente	Un vehículo puede no tener cliente registrado (entrada ocasional).
clientes — reservas	1 : N	reservas.id_cliente	Un cliente puede tener muchas reservas.
clientes — mensualidades	1 : N	mensualidades.id_cliente	Titular del plan mensual.
vehiculos — movimientos	1 : N	movimientos.id_vehiculo	Un vehículo acumula muchas estadías.
vehiculos — reservas	1 : N	reservas.id_vehiculo	Placa de la reserva.
vehiculos — mensualidades	1 : N	mensualidades.id_vehiculo	Placa cubierta por el plan.
espacios — movimientos	1 : N	movimientos.id_espacio	Espacio ocupado en cada estadía.
espacios — reservas	0..1 : N	reservas.id_espacio	Una reserva puede estar aún sin espacio asignado.
turnos — movimientos	1 : N (x2)	movimientos.id_turno_entrada, movimientos.id_turno_salida	Turno (y trabajador) que registró la entrada y la salida.
turnos — pagos	1 : N	pagos.id_turno	Recaudo del turno.
movimientos — pagos	1 : 0..1	pagos.id_movimiento	El cobro de la salida.
mensualidades — pagos	1 : 0..N	pagos.id_mensualidad	Pago de un plan mensual.
reservas — pagos	1 : 0..1	pagos.id_reserva	Cargo por reserva (RC-06).
reservas — alertas	1 : 0..N	alertas.id_reserva	Alerta de reserva por vencer.

6.5 Reglas de negocio derivadas de Figma

- Un movimiento con estado EN_PARQUEADERO y sin fecha de salida representa un vehículo dentro del parqueadero (RF-019).
- Solo se asigna un espacio DISPONIBLE del mismo tipo que el vehículo (RF-008).
- Al registrar la salida, el espacio vuelve a DISPONIBLE (RF-012).
- El valor de la estadía se calcula con las reglas de cobro RC-01 a RC-05 y la tarifa vigente del tipo de vehículo (RF-010, RF-017).
- El cambio es igual al efectivo recibido menos el valor; si el efectivo es menor, no se registra la operación (RF-011, RC-08).
- Cada pago pertenece a una sola operación: la salida (`id_movimiento`), el cargo de una reserva (`id_reserva`) o una mensualidad (`id_mensualidad`) (RC-06, RC-07).

- Un vehículo con mensualidad activa sale con valor \$0 y sin registro de pago (RC-05).
- La fecha de fin de una mensualidad es un mes después de la fecha de inicio, menos un día; si ese día no existe en el mes siguiente, es el último día de ese mes (RF-014, RC-07).
- Los estados se calculan con la fecha de fin: VENCIDA si ya pasó; "por vencer" si es ACTIVA y faltan 5 días o menos (RF-014, RC-10).

Elementos sin definir en el prototipo

- Recuperación de contraseña (RF-002): no se modela tabla porque el flujo no está diseñado.
- Alertas de sensor (RF-020): se modela el registro de la alerta, pero no la integración con sensores.
- Formato de exportación (RF-016): no requiere tabla.

7. Base de datos MySQL

Base de datos propuesta – pendiente de implementación

El código actual de PARKINLOCK **no contiene base de datos**. Este script es una **propuesta** que implementa en MySQL el modelo del documento 05. **No es código existente en el repositorio**. Los signos `?` representan parámetros que el backend debe enviar con consultas preparadas.

7.1 Creación de la base de datos, tablas e índices

```
-- =====
-- PARKINLOCK · Parqueadero Systems & Programación
-- BASE DE DATOS PROPUESTA - PENDIENTE DE IMPLEMENTACIÓN
-- MySQL 8. Diseñada a partir del prototipo de Figma (RF-001 a RF-021).
-- =====
CREATE DATABASE IF NOT EXISTS parkinlock_db
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE parkinlock_db;

CREATE TABLE roles (
  id_rol      INT AUTO_INCREMENT PRIMARY KEY,
  nombre      VARCHAR(30)  NOT NULL UNIQUE,
  descripcion VARCHAR(150) NULL
) ENGINE=InnoDB;

CREATE TABLE permisos (
  id_permiso  INT AUTO_INCREMENT PRIMARY KEY,
  id_rol      INT NOT NULL,
  area        ENUM('PANEL','ENTRADAS_SALIDAS','RESERVAS','MENSUALIDADES',
                  'HISTORIAL','TARIFAS_USUARIOS') NOT NULL,
  nivel_acceso VARCHAR(30) NOT NULL,
  CONSTRAINT uq_permiso UNIQUE (id_rol, area),
  CONSTRAINT fk_permisos_rol FOREIGN KEY (id_rol) REFERENCES roles(id_rol)
) ENGINE=InnoDB;

CREATE TABLE usuarios (
  id_usuario  INT AUTO_INCREMENT PRIMARY KEY,
  id_rol      INT NOT NULL,
  nombre_completo VARCHAR(120) NOT NULL,
  usuario     VARCHAR(50) NOT NULL UNIQUE,
  correo      VARCHAR(120) NULL UNIQUE,
  contrasena_hash VARCHAR(255) NOT NULL,
  estado      ENUM('ACTIVO','INACTIVO') NOT NULL DEFAULT 'ACTIVO',
  fecha_creacion DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_usuarios_rol FOREIGN KEY (id_rol) REFERENCES roles(id_rol)
) ENGINE=InnoDB;

CREATE TABLE turnos (
  id_turno    INT AUTO_INCREMENT PRIMARY KEY,
  id_usuario  INT NOT NULL,
  caja        VARCHAR(20) NOT NULL,
  inicio_programado DATETIME NOT NULL,
  fin_programado DATETIME NOT NULL,
  caja_inicial DECIMAL(12,2) NOT NULL DEFAULT 0,
  estado      ENUM('ABIERTO','CERRADO') NOT NULL DEFAULT 'ABIERTO',
  CONSTRAINT fk_turnos_usuario FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario),
  CONSTRAINT chk_turno_horas CHECK (fin_programado > inicio_programado)
) ENGINE=InnoDB;

CREATE TABLE tipos_vehiculo (
  id_tipo_vehiculo INT AUTO_INCREMENT PRIMARY KEY,
  nombre           VARCHAR(20) NOT NULL UNIQUE
) ENGINE=InnoDB;

CREATE TABLE espacios (
```

```

id_espacio      INT AUTO_INCREMENT PRIMARY KEY,
codigo         VARCHAR(10) NOT NULL UNIQUE,
id_tipo_vehiculo INT NOT NULL,
nivel         VARCHAR(20) NULL,
estado        ENUM('DISPONIBLE','OCUPADO','RESERVADO') NOT NULL DEFAULT 'DISPONIBLE',
CONSTRAINT fk_espacios_tipo FOREIGN KEY (id_tipo_vehiculo)
REFERENCES tipos_vehiculo(id_tipo_vehiculo)
) ENGINE=InnoDB;

CREATE TABLE clientes (
  id_cliente INT AUTO_INCREMENT PRIMARY KEY,
  nombre    VARCHAR(120) NOT NULL,
  telefono  VARCHAR(20) NULL
) ENGINE=InnoDB;

CREATE TABLE vehiculos (
  id_vehiculo INT AUTO_INCREMENT PRIMARY KEY,
  placa       VARCHAR(10) NOT NULL UNIQUE,
  id_tipo_vehiculo INT NOT NULL,
  id_cliente  INT NULL,
  CONSTRAINT fk_vehiculos_tipo FOREIGN KEY (id_tipo_vehiculo)
REFERENCES tipos_vehiculo(id_tipo_vehiculo),
  CONSTRAINT fk_vehiculos_cliente FOREIGN KEY (id_cliente)
REFERENCES clientes(id_cliente) ON DELETE SET NULL
) ENGINE=InnoDB;

CREATE TABLE tarifas (
  id_tarifa      INT AUTO_INCREMENT PRIMARY KEY,
  id_tipo_vehiculo INT NOT NULL,
  concepto       ENUM('HORA','FRACCION','NOCHE','DIA','RESERVA','MENSUALIDAD') NOT NULL,
  valor         DECIMAL(12,2) NOT NULL,
  vigente_desde  DATE NOT NULL,
  CONSTRAINT uq_tarifa UNIQUE (id_tipo_vehiculo, concepto, vigente_desde),
  CONSTRAINT chk_tarifa_valor CHECK (valor >= 0),
  CONSTRAINT fk_tarifas_tipo FOREIGN KEY (id_tipo_vehiculo)
REFERENCES tipos_vehiculo(id_tipo_vehiculo)
) ENGINE=InnoDB;

CREATE TABLE movimientos (
  id_movimiento INT AUTO_INCREMENT PRIMARY KEY,
  id_vehiculo   INT NOT NULL,
  id_espacio    INT NOT NULL,
  id_turno_entrada INT NOT NULL,
  id_turno_salida INT NULL,
  fecha_hora_entrada DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  fecha_hora_salida DATETIME NULL,
  minutos        INT NULL,
  valor_total    DECIMAL(12,2) NULL,
  estado        ENUM('EN_PARQUEADERO','FINALIZADO') NOT NULL DEFAULT 'EN_PARQUEADERO',
  CONSTRAINT fk_mov_vehiculo FOREIGN KEY (id_vehiculo) REFERENCES vehiculos(id_vehiculo),
  CONSTRAINT fk_mov_espacio FOREIGN KEY (id_espacio) REFERENCES espacios(id_espacio),
  CONSTRAINT fk_mov_turno_in FOREIGN KEY (id_turno_entrada) REFERENCES turnos(id_turno),
  CONSTRAINT fk_mov_turno_out FOREIGN KEY (id_turno_salida) REFERENCES turnos(id_turno),
  CONSTRAINT chk_mov_fechas CHECK (fecha_hora_salida IS NULL
OR fecha_hora_salida >= fecha_hora_entrada)
) ENGINE=InnoDB;

CREATE TABLE reservas (
  id_reserva INT AUTO_INCREMENT PRIMARY KEY,
  id_vehiculo INT NOT NULL,
  id_cliente  INT NOT NULL,
  id_espacio  INT NULL,
  id_usuario  INT NOT NULL,
  fecha      DATE NOT NULL,
  hora_inicio TIME NOT NULL,
  hora_fin   TIME NOT NULL,
  estado     ENUM('PENDIENTE','CONFIRMADA') NOT NULL DEFAULT 'PENDIENTE',
  CONSTRAINT fk_res_vehiculo FOREIGN KEY (id_vehiculo) REFERENCES vehiculos(id_vehiculo),
  CONSTRAINT fk_res_cliente FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente),
  CONSTRAINT fk_res_espacio FOREIGN KEY (id_espacio) REFERENCES espacios(id_espacio),
  CONSTRAINT fk_res_usuario FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario),

```

```

CONSTRAINT chk_res_horas CHECK (hora_fin > hora_inicio)
) ENGINE=InnoDB;

CREATE TABLE mensualidades (
  id_mensualidad INT AUTO_INCREMENT PRIMARY KEY,
  id_vehiculo    INT NOT NULL,
  id_cliente     INT NOT NULL,
  fecha_inicio  DATE NOT NULL,
  fecha_fin     DATE NOT NULL,
  valor         DECIMAL(12,2) NOT NULL,
  estado        ENUM('ACTIVA','VENCIDA') NOT NULL DEFAULT 'ACTIVA',
  CONSTRAINT fk_men_vehiculo FOREIGN KEY (id_vehiculo) REFERENCES vehiculos(id_vehiculo),
  CONSTRAINT fk_men_cliente FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente),
  CONSTRAINT chk_men_fechas CHECK (fecha_fin >= fecha_inicio)
) ENGINE=InnoDB;

CREATE TABLE pagos (
  id_pago        INT AUTO_INCREMENT PRIMARY KEY,
  id_movimiento  INT NULL UNIQUE,
  id_reserva     INT NULL UNIQUE,
  id_mensualidad INT NULL,
  id_turno       INT NOT NULL,
  metodo         ENUM('EFECTIVO') NOT NULL DEFAULT 'EFECTIVO',
  valor         DECIMAL(12,2) NOT NULL,
  efectivo_recibido DECIMAL(12,2) NOT NULL,
  cambio         DECIMAL(12,2) NOT NULL,
  fecha_hora     DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_pag_mov FOREIGN KEY (id_movimiento) REFERENCES movimientos(id_movimiento),
  CONSTRAINT fk_pag_res FOREIGN KEY (id_reserva) REFERENCES reservas(id_reserva),
  CONSTRAINT fk_pag_men FOREIGN KEY (id_mensualidad) REFERENCES mensualidades(id_mensualidad),
  CONSTRAINT fk_pag_turno FOREIGN KEY (id_turno) REFERENCES turnos(id_turno),
  CONSTRAINT chk_pag_efectivo CHECK (efectivo_recibido >= valor),
  CONSTRAINT chk_pag_cambio CHECK (cambio = efectivo_recibido - valor),
  CONSTRAINT chk_pag_origen CHECK ((id_movimiento IS NOT NULL) + (id_reserva IS NOT NULL)
                                   + (id_mensualidad IS NOT NULL) = 1)
) ENGINE=InnoDB;

CREATE TABLE alertas (
  id_alerta INT AUTO_INCREMENT PRIMARY KEY,
  tipo      ENUM('RESERVA_POR_VENCER','SENSOR') NOT NULL,
  mensaje   VARCHAR(200) NOT NULL,
  id_reserva INT NULL,
  fecha_hora DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  estado     ENUM('ACTIVA','ATENDIDA') NOT NULL DEFAULT 'ACTIVA',
  CONSTRAINT fk_alerta_reserva FOREIGN KEY (id_reserva) REFERENCES reservas(id_reserva)
) ENGINE=InnoDB;

-- Índices para las consultas principales
CREATE INDEX idx_mov_estado ON movimientos (estado);
CREATE INDEX idx_mov_entrada ON movimientos (fecha_hora_entrada);
CREATE INDEX idx_mov_vehiculo ON movimientos (id_vehiculo, estado);
CREATE INDEX idx_res_fecha ON reservas (fecha, estado);
CREATE INDEX idx_men_fin ON mensualidades (fecha_fin, estado);
CREATE INDEX idx_pag_fecha ON pagos (fecha_hora);
CREATE INDEX idx_espacio_estado ON espacios (estado, id_tipo_vehiculo);

```

7.2 Resumen de restricciones

Tipo	Aplicación
PRIMARY KEY	14 tablas con llave primaria entera AUTO_INCREMENT.
FOREIGN KEY	Relaciones del documento 05 (22 llaves foráneas).

Tipo	Aplicación
UNIQUE	roles.nombre, usuarios.usuario, usuarios.correo, tipos_vehiculo.nombre, espacios.codigo, vehiculos.placa, (rol, área), (tipo, concepto, vigencia), pagos.id_movimiento, pagos.id_reserva.
CHECK	Horas de turno y reserva, fechas de movimiento y mensualidad, tarifa no negativa, efectivo suficiente, cambio correcto y un único origen por pago (salida, reserva o mensualidad).
ENUM	Estados de usuario, turno, espacio, movimiento, reserva, mensualidad y alerta; conceptos de tarifa; áreas de permiso.
INDEX	7 índices para panel, mapa, historial, reservas, mensualidades y recaudo.

7.3 Datos mínimos de prueba

```

-----
-- Datos mínimos de prueba (FICTICIOS). Las tarifas y permisos
-- corresponden a los valores mostrados en el prototipo de Figma.
-- contrasena_hash: reemplazar por hashes generados por el backend.
-----
INSERT INTO roles (nombre, descripcion) VALUES
('ADMINISTRADOR', 'Configura tarifas, usuarios y permisos'),
('TRABAJADOR', 'Opera entradas, salidas y cobros');

INSERT INTO permisos (id_rol, area, nivel_acceso) VALUES
(1, 'PANEL', 'Ver y editar'), (2, 'PANEL', 'Ver'),
(1, 'ENTRADAS_SALIDAS', 'Gestionar'), (2, 'ENTRADAS_SALIDAS', 'Gestionar'),
(1, 'RESERVAS', 'Gestionar'), (2, 'RESERVAS', 'Crear y ver'),
(1, 'MENSUALIDADES', 'Gestionar'), (2, 'MENSUALIDADES', 'Solo ver'),
(1, 'HISTORIAL', 'Ver y exportar'), (2, 'HISTORIAL', 'Ver turno propio'),
(1, 'TARIFAS_USUARIOS', 'Configurar'), (2, 'TARIFAS_USUARIOS', 'Sin acceso');

INSERT INTO tipos_vehiculo (nombre) VALUES ('CARRO'), ('MOTO');

INSERT INTO tarifas (id_tipo_vehiculo, concepto, valor, vigente_desde) VALUES
(1, 'HORA', 5000, '2026-10-01'), (2, 'HORA', 3000, '2026-10-01'),
(1, 'FRACCION', 5000, '2026-10-01'), (2, 'FRACCION', 3000, '2026-10-01'),
(1, 'NOCHE', 25000, '2026-10-01'), (2, 'NOCHE', 15000, '2026-10-01'),
(1, 'DIA', 35000, '2026-10-01'), (2, 'DIA', 22000, '2026-10-01'),
(1, 'RESERVA', 8000, '2026-10-01'), (2, 'RESERVA', 5000, '2026-10-01'),
(1, 'MENSUALIDAD', 180000, '2026-10-01'), (2, 'MENSUALIDAD', 95000, '2026-10-01');

INSERT INTO espacios (codigo, id_tipo_vehiculo) VALUES
('C-01', 1), ('C-02', 1), ('C-12', 1), ('M-01', 2), ('M-02', 2);

INSERT INTO usuarios (id_rol, nombre_completo, usuario, contrasena_hash) VALUES
(1, 'Administrador de prueba', 'admin', '<hash>'),
(2, 'Trabajador de prueba', 'trabajador', '<hash>');

```

7.4 Consultas principales propuestas

7.4.1 1. Inicio de sesión: usuario por nombre de usuario o correo y rol (RF-001, RF-004)

```

SELECT u.id_usuario, u.nombre_completo, u.contrasena_hash, u.estado, r.nombre AS rol
FROM usuarios u JOIN roles r ON r.id_rol = u.id_rol
WHERE (u.usuario = ? OR u.correo = ?) AND r.nombre = ?;

```

7.4.2 2. Permisos del rol (RF-004)

```

SELECT area, nivel_acceso FROM permisos WHERE id_rol = ?;

```

7.4.3 3. Indicadores del panel (RF-005)

```
SELECT
  (SELECT COUNT(*) FROM movimientos WHERE estado = 'EN_PARQUEADERO') AS vehiculos_dentro,
  (SELECT COUNT(*) FROM espacios WHERE estado = 'DISPONIBLE') AS disponibles,
  (SELECT COUNT(*) FROM espacios WHERE estado = 'OCUPADO') AS ocupados,
  (SELECT COUNT(*) FROM espacios WHERE estado = 'RESERVADO') AS reservados,
  (SELECT COALESCE(SUM(valor),0) FROM pagos WHERE DATE(fecha_hora) = CURDATE()) AS recaudado_hoy;
```

7.4.4 4. Mapa de espacios con placa del vehículo (RF-006)

```
SELECT e.codigo, t.nombre AS tipo, e.estado, v.placa
FROM espacios e
JOIN tipos_vehiculo t ON t.id_tipo_vehiculo = e.id_tipo_vehiculo
LEFT JOIN movimientos m ON m.id_espacio = e.id_espacio AND m.estado = 'EN_PARQUEADERO'
LEFT JOIN vehiculos v ON v.id_vehiculo = m.id_vehiculo
ORDER BY e.codigo;
```

7.4.5 5. Registrar entrada (RF-008)

```
START TRANSACTION;
INSERT INTO movimientos (id_vehiculo, id_espacio, id_turno_entrada) VALUES (?, ?, ?);
UPDATE espacios SET estado = 'OCUPADO' WHERE id_espacio = ? AND estado = 'DISPONIBLE';
COMMIT;
```

7.4.6 6. Buscar vehículo por placa para la salida (RF-009)

```
SELECT m.id_movimiento, v.placa, t.nombre AS tipo, e.codigo AS espacio, m.fecha_hora_entrada,
  TIMESTAMPDIFF(MINUTE, m.fecha_hora_entrada, NOW()) AS minutos
FROM movimientos m
JOIN vehiculos v ON v.id_vehiculo = m.id_vehiculo
JOIN tipos_vehiculo t ON t.id_tipo_vehiculo = v.id_tipo_vehiculo
JOIN espacios e ON e.id_espacio = m.id_espacio
WHERE v.placa = ? AND m.estado = 'EN_PARQUEADERO';
```

7.4.7 7. Tarifas vigentes de un tipo de vehículo (RF-010, RF-017)

```
SELECT t.concepto, t.valor
FROM tarifas t
WHERE t.id_tipo_vehiculo = ?
  AND t.vigente_desde = (SELECT MAX(t2.vigente_desde) FROM tarifas t2
    WHERE t2.id_tipo_vehiculo = t.id_tipo_vehiculo
    AND t2.concepto = t.concepto
    AND t2.vigente_desde <= CURDATE());
```

7.4.8 8. Registrar pago, salida y liberar espacio (RF-011, RF-012)

```
START TRANSACTION;
UPDATE movimientos
  SET fecha_hora_salida = NOW(), id_turno_salida = ?, minutos = ?, valor_total = ?, estado = 'FINALIZADO'
  WHERE id_movimiento = ? AND estado = 'EN_PARQUEADERO';
INSERT INTO pagos (id_movimiento, id_turno, valor, efectivo_recibido, cambio)
VALUES (?, ?, ?, ?, ?);
UPDATE espacios SET estado = 'DISPONIBLE' WHERE id_espacio = ?;
COMMIT;
```

7.4.9 9. Salida de un vehículo con mensualidad activa, sin pago (RF-012, RC-05)

```
START TRANSACTION;
UPDATE movimientos m
  SET m.fecha_hora_salida = NOW(), m.id_turno_salida = ?, m.minutos = ?, m.valor_total = 0, m.estado = 'FINALIZADO'
 WHERE m.id_movimiento = ? AND m.estado = 'EN_PARQUEADERO'
  AND EXISTS (SELECT 1 FROM mensualidades me
              WHERE me.id_vehiculo = m.id_vehiculo AND me.estado = 'ACTIVA'
              AND CURDATE() BETWEEN me.fecha_inicio AND me.fecha_fin);
UPDATE espacios SET estado = 'DISPONIBLE' WHERE id_espacio = ?;
COMMIT;
```

7.4.10 10. Confirmar reserva y cobrar el cargo (RF-013, RC-06)

```
START TRANSACTION;
UPDATE reservas SET estado = 'CONFIRMADA' WHERE id_reserva = ? AND estado = 'PENDIENTE';
INSERT INTO pagos (id_reserva, id_turno, valor, efectivo_recibido, cambio)
VALUES (?, ?, ?, ?, ?);
COMMIT;
```

7.4.11 11. Crear mensualidad de un mes y registrar su pago (RF-014, RC-07)

```
SET @inicio = ?;
START TRANSACTION;
INSERT INTO mensualidades (id_vehiculo, id_cliente, fecha_inicio, fecha_fin, valor)
VALUES (?, ?, @inicio,
        CASE WHEN DAY(DATE_ADD(@inicio, INTERVAL 1 MONTH)) < DAY(@inicio)
              THEN DATE_ADD(@inicio, INTERVAL 1 MONTH)
              ELSE DATE_SUB(DATE_ADD(@inicio, INTERVAL 1 MONTH), INTERVAL 1 DAY) END,
        ?);
INSERT INTO pagos (id_mensualidad, id_turno, valor, efectivo_recibido, cambio)
VALUES (LAST_INSERT_ID(), ?, ?, ?, ?);
COMMIT;
```

7.4.12 12. Reservas del día (RF-013)

```
SELECT v.placa, c.nombre AS cliente, r.hora_inicio, r.hora_fin, e.codigo AS espacio, r.estado
FROM reservas r
JOIN vehiculos v ON v.id_vehiculo = r.id_vehiculo
JOIN clientes c ON c.id_cliente = r.id_cliente
LEFT JOIN espacios e ON e.id_espacio = r.id_espacio
WHERE r.fecha = CURDATE() ORDER BY r.hora_inicio;
```

7.4.13 13. Mensualidades con estado calculado (RF-014, RC-07, RC-10)

```
SELECT v.placa, c.nombre AS titular, CONCAT(t.nombre, ' mensual') AS plan,
       m.fecha_inicio, m.fecha_fin, m.valor,
       CASE WHEN m.fecha_fin < CURDATE()
             WHEN DATEDIFF(m.fecha_fin, CURDATE()) <= 5
             ELSE 'ACTIVA' END AS estado
       THEN 'VENCIDA'
       THEN 'POR VENCER'
FROM mensualidades m
JOIN vehiculos v ON v.id_vehiculo = m.id_vehiculo
JOIN tipos_vehiculo t ON t.id_tipo_vehiculo = v.id_tipo_vehiculo
JOIN clientes c ON c.id_cliente = m.id_cliente;
```

7.4.14 14. Historial de movimientos con filtros (RF-015, RF-016)

```
SELECT v.placa, DATE(m.fecha_hora_entrada) AS fecha, TIME(m.fecha_hora_entrada) AS entrada,
       TIME(m.fecha_hora_salida) AS salida, m.minutos, p.valor AS valor_pagado,
       u.nombre_completo AS trabajador
```

```
FROM movimientos m
JOIN vehiculos v ON v.id_vehiculo = m.id_vehiculo
JOIN turnos tu ON tu.id_turno = m.id_turno_entrada
JOIN usuarios u ON u.id_usuario = tu.id_usuario
LEFT JOIN pagos p ON p.id_movimiento = m.id_movimiento
WHERE (? IS NULL OR v.placa = ?) AND DATE(m.fecha_hora_entrada) = ?
ORDER BY m.fecha_hora_entrada DESC;
```

7.4.15 15. Vehículos en operación (RF-019)

```
SELECT v.placa, c.nombre AS cliente, t.nombre AS tipo, m.fecha_hora_entrada,
       TIMESTAMPDIFF(MINUTE, m.fecha_hora_entrada, NOW()) AS permanencia_min, e.codigo AS espacio
FROM movimientos m
JOIN vehiculos v ON v.id_vehiculo = m.id_vehiculo
JOIN tipos_vehiculo t ON t.id_tipo_vehiculo = v.id_tipo_vehiculo
JOIN espacios e ON e.id_espacio = m.id_espacio
LEFT JOIN clientes c ON c.id_cliente = v.id_cliente
WHERE m.estado = 'EN_PARQUEADERO'
ORDER BY m.fecha_hora_entrada LIMIT ? OFFSET ?;
```

7.4.16 16. Alertas activas (RF-020)

```
SELECT tipo, mensaje, fecha_hora FROM alertas WHERE estado = 'ACTIVA' ORDER BY fecha_hora DESC;
```

7.4.17 17. Turno activo del usuario (RF-021)

```
SELECT caja, inicio_programado, fin_programado, caja_inicial, estado
FROM turnos WHERE id_usuario = ? AND estado = 'ABIERTO';
```

8. Arquitectura del sistema

PARKINLOCK sigue una arquitectura cliente-servidor de tres capas: **Frontend** → **Backend/API** → **Base de datos**. Este documento separa lo que existe hoy de la arquitectura propuesta para implementar lo diseñado en Figma.

8.1 Arquitectura actualmente existente EXISTENTE EN CÓDIGO

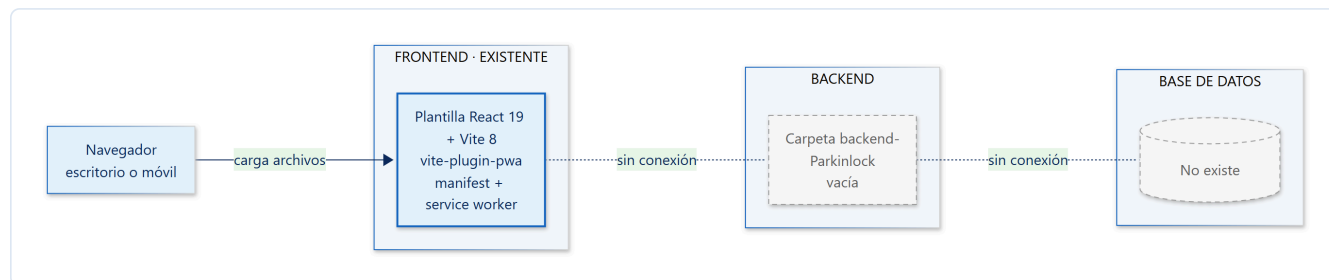


Figura 2. Arquitectura actual: solo existe el frontend base.

Capa	Estado actual verificado
Frontend	Proyecto React 19.2 + Vite 8.3 en <code>frontend-Parkinlock-</code> . Muestra la página de ejemplo de la plantilla. Configurado como PWA con vite-plugin-pwa 1.3 (manifest, service worker Workbox con actualización automática e íconos).
Backend / API	La carpeta <code>backend-Parkinlock</code> existe solo en el equipo local, está vacía y no está registrada en Git.
Base de datos	No existe. No hay archivos SQL ni configuración de conexión.
Comunicación	No hay comunicación entre capas: el frontend no realiza peticiones a ningún servidor.

8.2 Arquitectura funcional propuesta PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN

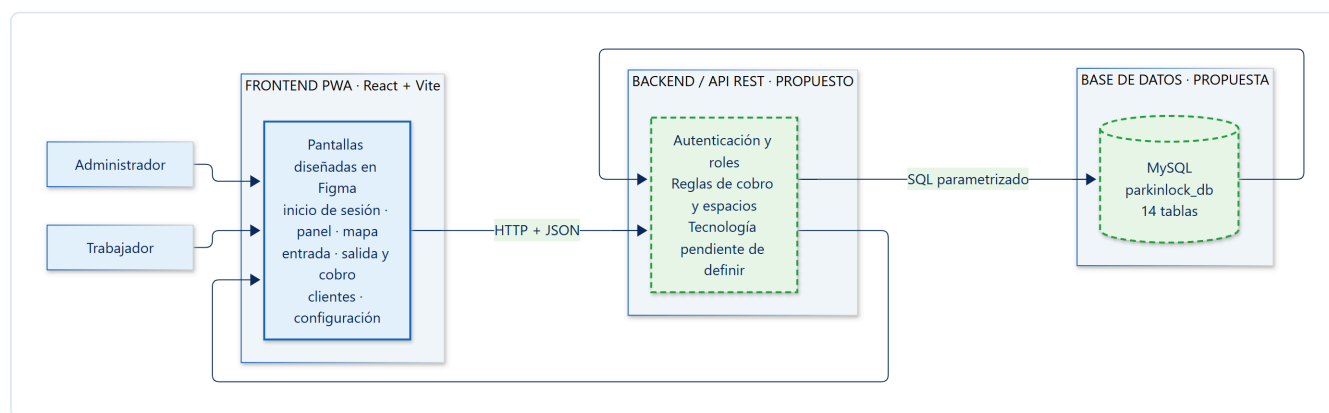


Figura 3. Arquitectura propuesta para PARKINLOCK (pendiente de implementación).

8.2.1 Frontend

Se mantiene la tecnología existente (React + Vite + vite-plugin-pwa). Debe implementar las 11 pantallas del prototipo de Figma para escritorio y móvil, consumir la API y mostrar los menús según el rol.

8.2.2 Backend / API

Servicio que expone una API REST en formato JSON y concentra las reglas del negocio: autenticación, permisos por rol, asignación de espacios, cálculo de tiempo y valor, pagos, reservas, mensualidades, historial y alertas. **La tecnología del backend no está definida en el proyecto [Pendiente de definir por el equipo]**; esta documentación no asume ningún framework.

8.2.3 Base de datos

Base de datos relacional MySQL propuesta en los documentos 05 y 06 (14 tablas). Pendiente de implementación.

8.3 Comunicación entre componentes (propuesta)

1. El Administrador o el Trabajador usa la PWA desde el navegador o instalada en su dispositivo.
2. El frontend envía peticiones HTTP con datos JSON a la API, incluyendo la identificación de la sesión.
3. La API valida el rol, aplica las reglas del negocio y ejecuta consultas SQL parametrizadas.
4. MySQL devuelve los datos a la API, que responde al frontend en JSON.
5. El frontend actualiza el panel, el mapa y los indicadores.

8.4 Flujo de información: entrada, cobro y salida

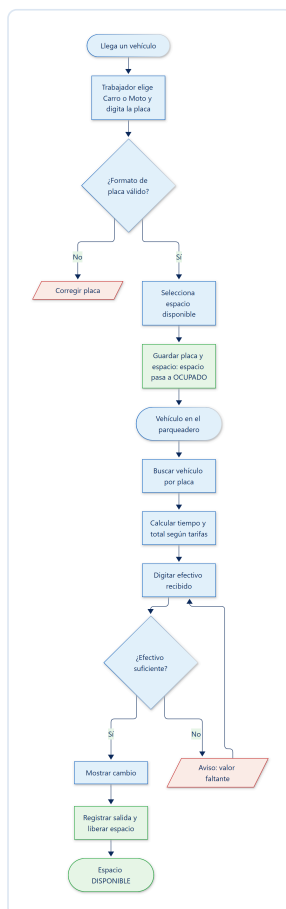


Figura 4. Flujo operativo diseñado en Figma (pendiente de implementación).

9. Tecnologías y documentación técnica

9.1 Estructura real del repositorio EXISTENTE EN CÓDIGO

```
Parkinlock-/
├── .gitignore           Exclusiones de Git (node_modules, .env, dist, ...)
├── LICENSE             Licencia Apache 2.0
├── README.md           Descripción del propósito del parqueadero
├── DOCUMENTACION_PARQUEADERO/ Documentación del proyecto
├── backend-Parkinlock/ Vacía (solo local, no registrada en Git)
├── frontend-Parkinlock-/
│   ├── index.html      HTML principal: íconos, color de tema, título y #root
│   ├── package.json     Dependencias y scripts
│   ├── package-lock.json Versiones exactas instaladas
│   ├── vite.config.js   Plugins de React y PWA, manifest
│   ├── pwa-assets.config.js Generación de íconos desde public/favicon.svg
│   ├── .oxlintrc.json   Reglas del linter (react, oxc)
│   ├── .gitignore       Exclusiones del frontend
│   ├── README.md        README de la plantilla de Vite
│   ├── public/           favicon.svg, favicon.ico, icons.svg e íconos PWA
│   └── src/
│       ├── main.jsx     Punto de entrada: renderiza <App /> en #root
│       ├── App.jsx       Página de ejemplo de la plantilla (contador)
│       ├── App.css, index.css Estilos de la plantilla
│       └── assets/       hero.png, react.svg, vite.svg
```

9.2 Archivos principales

Archivo	Qué contiene actualmente	Qué falta
src/App.jsx	Página de ejemplo con logos de Vite y React, un botón contador y enlaces a documentación.	Reemplazar por las pantallas de PARKINLOCK y su navegación.
src/main.jsx	Monta el componente App en modo estricto de React.	Agregar el enrutamiento entre pantallas.
package.json	Dependencias: react 19.2, react-dom 19.2. Desarrollo: vite 8.3, @vitejs/plugin-react 6.1, vite-plugin-pwa 1.3, @vite-pwa/assets-generator 1.0.4, oxlint 1.81.	Dependencias que se definan para enrutamiento y consumo de la API.
vite.config.js	Plugin de React y VitePWA con registerType "autoUpdate", manifest con nombre "Parkinlock", colores #ffffff, modo standalone e íconos.	Ajustar colores del manifest a la identidad del logo.
index.html	Enlaces a favicon, apple-touch-icon, theme-color y título "Parkinlock".	—
pwa-assets.config.js	Preset minimal-2023 tomando public/favicon.svg.	Reemplazar favicon.svg por el logo oficial y regenerar íconos.
README.md (raíz)	Propósito del parqueadero y modelo de cobro.	Instrucciones de instalación y uso.
backend-Parkinlock/	Carpeta vacía.	Todo el backend.

9.3 Tecnologías detectadas en el código

Tecnología	Versión	Uso
React / React DOM	19.2	Librería de interfaz.
Vite	8.3	Servidor de desarrollo y compilación.

Tecnología	Versión	Uso
@vitejs/plugin-react	6.1	Soporte de React en Vite.
vite-plugin-pwa (Workbox)	1.3	Manifest y service worker.
@vite-pwa/assets-generator	1.0.4	Generación de íconos PWA.
oxlint	1.81	Linter.
JavaScript (JSX), HTML, CSS	—	Lenguajes del frontend.
Node.js y npm	—	Entorno de ejecución y gestor de paquetes.
Git y GitHub	—	Control de versiones.

MySQL no aparece en el código; se incluye únicamente como base de datos propuesta en los documentos 05 y 06.

9.4 Requisitos para ejecutar

- Node.js (versión LTS) y npm.
- Git.
- Navegador actualizado.

9.5 Instalación y ejecución local

```
git clone https://github.com/camilotobon34-sudo/Parkinlock-.git
cd Parkinlock-/frontend-Parkinlock-
npm install
npm run dev # http://localhost:5173
```

Script	Comando	Descripción
dev	<code>npm run dev</code>	Servidor de desarrollo.
build	<code>npm run build</code>	Compila en dist/ con manifest y service worker.
preview	<code>npm run preview</code>	Sirve el build localmente.
lint	<code>npm run lint</code>	Ejecuta oxlint.
generate-pwa-assets	<code>npm run generate-pwa-assets</code>	Regenera los íconos PWA.

Importante

Los comandos de npm deben ejecutarse dentro de `frontend-Parkinlock-`. Al ejecutarlos en la raíz se crea un `package.json` incorrecto.

9.6 Estado por componente

Componente	Estado	Detalle
Frontend	EXISTENTE EN CÓDIGO	Base técnica lista (React, Vite, PWA). Pantallas de PARKINLOCK: 0 de 11 construidas.
Backend	NO INICIADO	Sin código, sin tecnología definida.

Componente	Estado	Detalle
Base de datos	PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN	Modelo y script propuestos; no existe en el código.
Despliegue	NO REALIZADO	Sin URL de producción.

9.7 Pendientes de desarrollo

#	Pendiente	Requisitos
1	Definir la tecnología del backend y crear su estructura.	—
2	Crear la base de datos MySQL con el script del documento 06.	Todos
3	Implementar autenticación, roles y permisos.	RF-001 a RF-004, RF-018
4	Construir el panel, el mapa y los movimientos.	RF-005 a RF-007, RF-019 a RF-021
5	Construir entrada, salida, cobro y pago.	RF-008 a RF-012
6	Construir reservas, mensualidades e historial con exportación.	RF-013 a RF-016
7	Construir la configuración de tarifas.	RF-017
8	Diseñar las pantallas faltantes: recuperación de contraseña, nueva reserva, nueva mensualidad, gestión de usuarios.	RF-002, RF-013, RF-014, RF-018
9	Reemplazar favicon.svg por el logo oficial y ajustar colores del manifest.	RNF-005
10	Desplegar la aplicación.	RNF-014

10. Manual de usuario del diseño funcional

Manual del diseño funcional

Este manual describe **cómo se usará PARKINLOCK según el prototipo de Figma**. Las funciones descritas están **diseñadas y pendientes de implementación**; la aplicación actual todavía no las incluye. Las imágenes son pantallas del prototipo.

Algunas pantallas del prototipo muestran un nombre de marca provisional. El nombre oficial del proyecto es **PARKINLOCK**.

10.1 Ingreso al sistema y selección de rol

1. Abra PARKINLOCK en el navegador o desde el ícono instalado **[URL pendiente]**.
2. Seleccione su rol: **Administrador** o **Trabajador**.
3. Escriba su **correo o usuario** y su **contraseña**. Puede mostrar la contraseña con el ícono de ojo.
4. Opcional: marque **Recordar acceso**.
5. Pulse **Entrar al sistema** (en móvil, **Entrar**).

Si olvidó su clave, pulse **¿Olvidaste tu clave?** o contacte al administrador de la sede (el flujo de recuperación está pendiente de diseño).

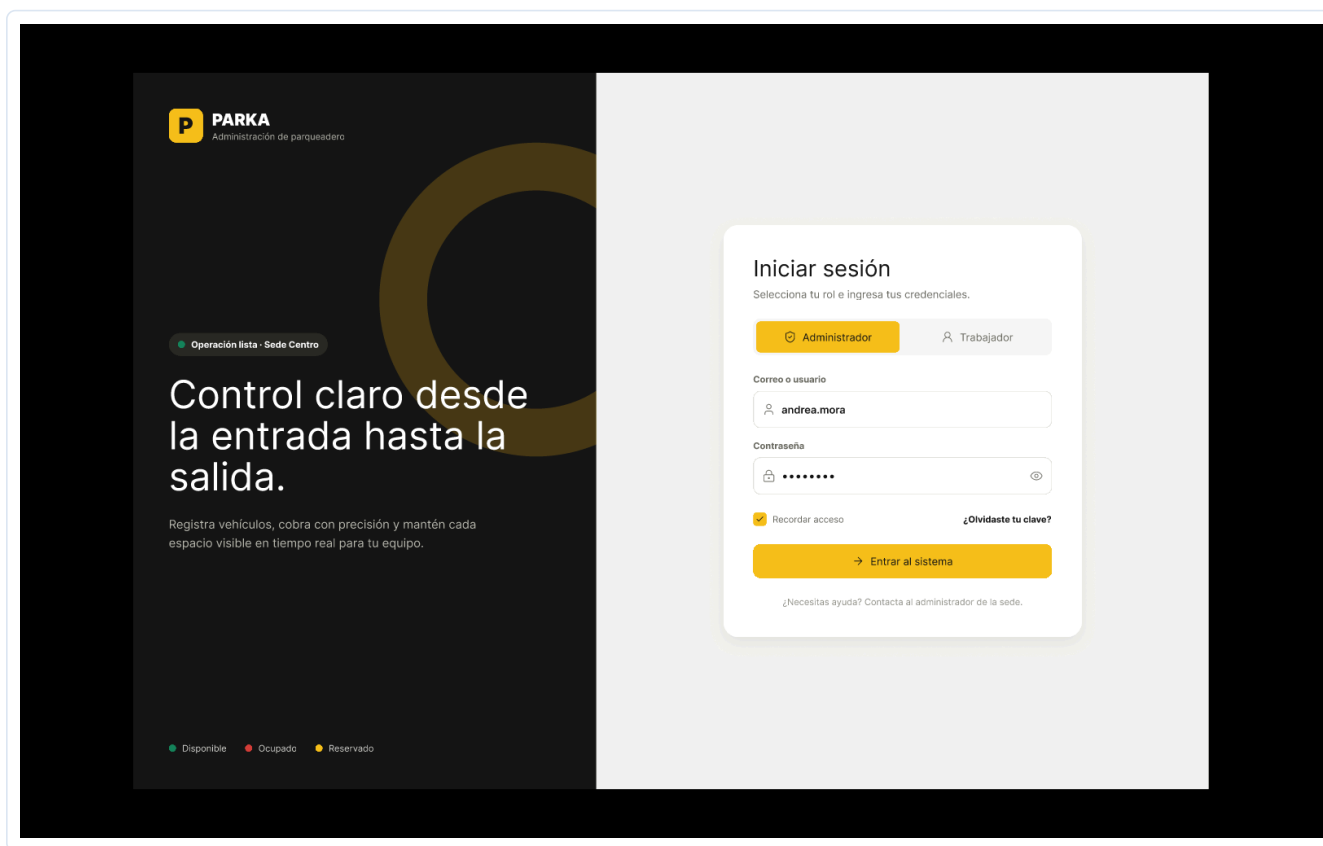


Figura 5. Pantalla F01 del prototipo – Inicio de sesión (escritorio).

10.2 Panel general

Muestra los indicadores del día: vehículos dentro, espacios disponibles, ocupados y reservados, y lo recaudado. También el porcentaje de ocupación, los últimos movimientos (con el enlace **Ver historial**) y su turno activo. Desde el panel puede ir directamente a **Registrar entrada**, **Cobrar salida** o **Nueva reserva**.

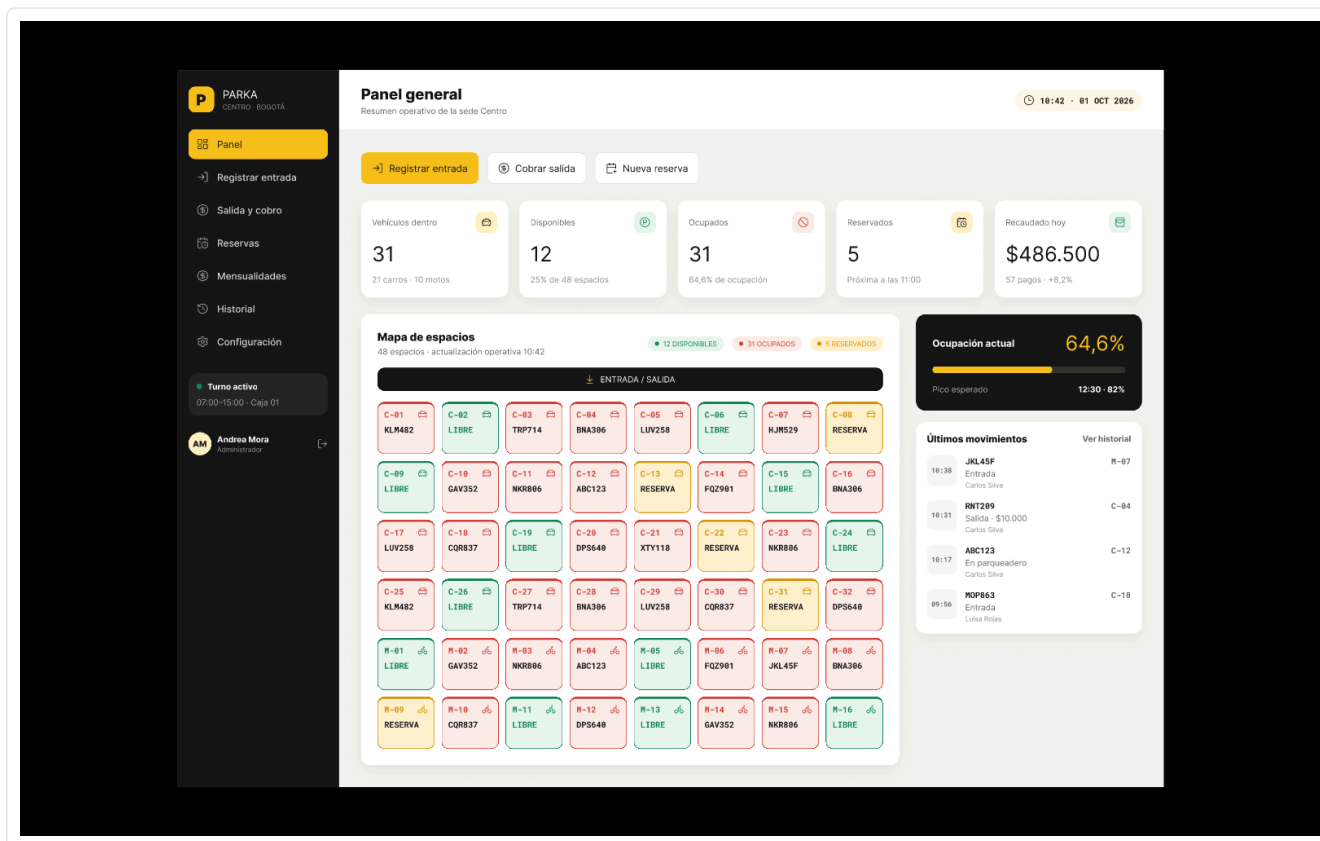


Figura 6. Pantalla F02 del prototipo – Panel general (escritorio, rol Administrador).

10.3 Mapa de espacios

Cada recuadro es un espacio: los que empiezan por **C** son para carro y los que empiezan por **M** para moto. El estado se indica con color y texto: **Libre** (verde), **Ocupado** (rojo, con la placa) o **Reserva** (amarillo). En móvil se muestra un mapa rápido con la opción **Ver todos**.

10.4 Registrar la entrada de un vehículo

1. Pulse **Registrar entrada** en el menú o en el panel.
2. Seleccione el tipo: **Carro** o **Moto**.
3. Escriba la **placa**. El sistema valida el formato.
4. Seleccione el **espacio** (o uno de los espacios cercanos disponibles). La hora de entrada se toma automáticamente.
5. Verifique el mensaje de disponibilidad y pulse **Guardar placa y espacio**. Para salir sin guardar, pulse **Cancelar**.

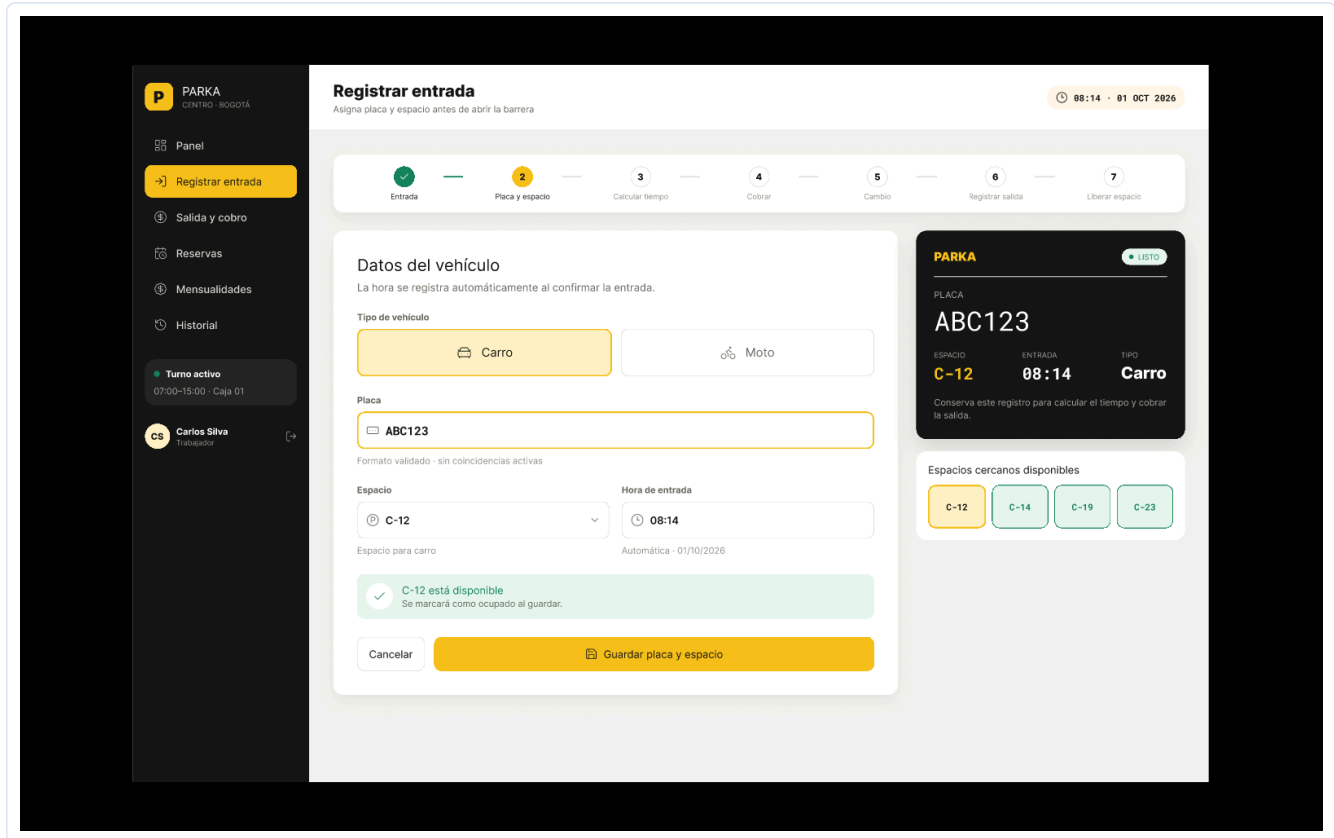


Figura 7. Pantalla F04 del prototipo – Registrar entrada (escritorio, rol Trabajador).

10.5 Buscar el vehículo, cobrar y registrar la salida

1. Pulse **Salida y cobro**.
2. Escriba la placa en **Buscar vehículo por placa** y pulse **Buscar**. Se muestra la hora de entrada, la hora de salida y el tiempo utilizado.
3. Revise el **cálculo del cobro** (horas y fracción) y el **Total a pagar**. El cálculo sigue las reglas de cobro del documento 02 (RC-01 a RC-05).
4. Digite el **Efectivo recibido**. El sistema muestra el **Cambio para el cliente** o el valor que falta.
5. Revise el resumen y pulse **Registrar salida y liberar [espacio]**. El espacio queda disponible.

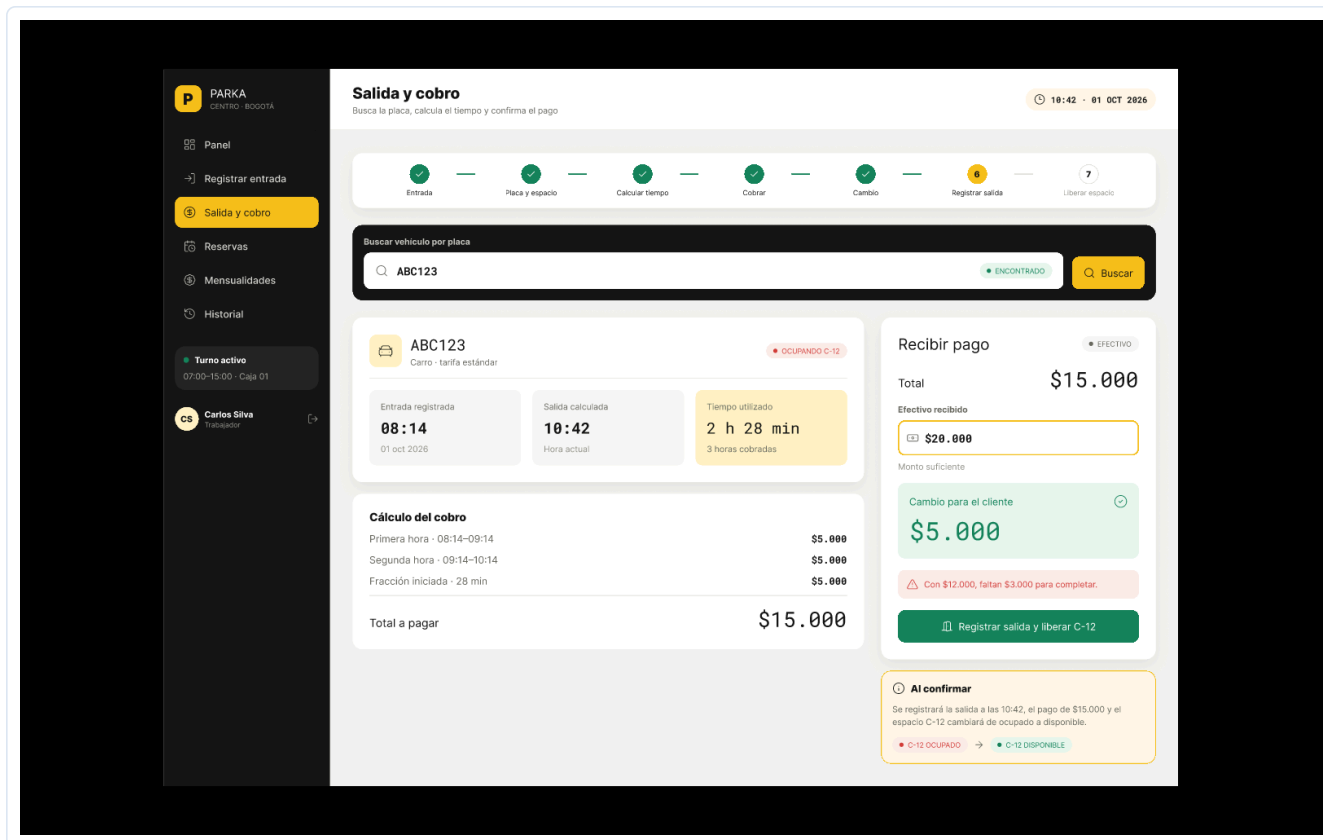


Figura 8. Pantalla F05 del prototipo – Salida y cobro (escritorio).

10.6 Reservas

En **Reservas** (pantalla Gestión de clientes, pestaña Reservas) se ven las reservas de hoy con placa, cliente, horario, espacio y estado (Confirmada o Pendiente). Pulse **Nueva reserva** para crear una (formulario pendiente de diseño). El Trabajador puede crear y ver reservas; el Administrador puede gestionarl

10.7 Mensualidades

En la pestaña **Mensualidades** se listan placa, titular, plan, vigencia, valor y estado (Activa o Vencida). Cada plan dura un mes desde su fecha de inicio (por ejemplo, del 01 al 31 de octubre o del 15 de octubre al 14 de noviembre) y se cuenta como "por vencer" cuando le faltan 5 días o menos. El Administrador puede crear planes con **Nueva mensualidad** (formulario pendiente de diseño); el Trabajador solo puede consultarlos.

10.8 Historial

En la pestaña **Historial** se consultan las entradas, salidas y pagos: placa, fecha, entrada, salida, tiempo, valor pagado y trabajador. Puede buscar por placa y filtrar por fecha. El Administrador puede pulsar **Exportar**; el Trabajador ve solo su turno.

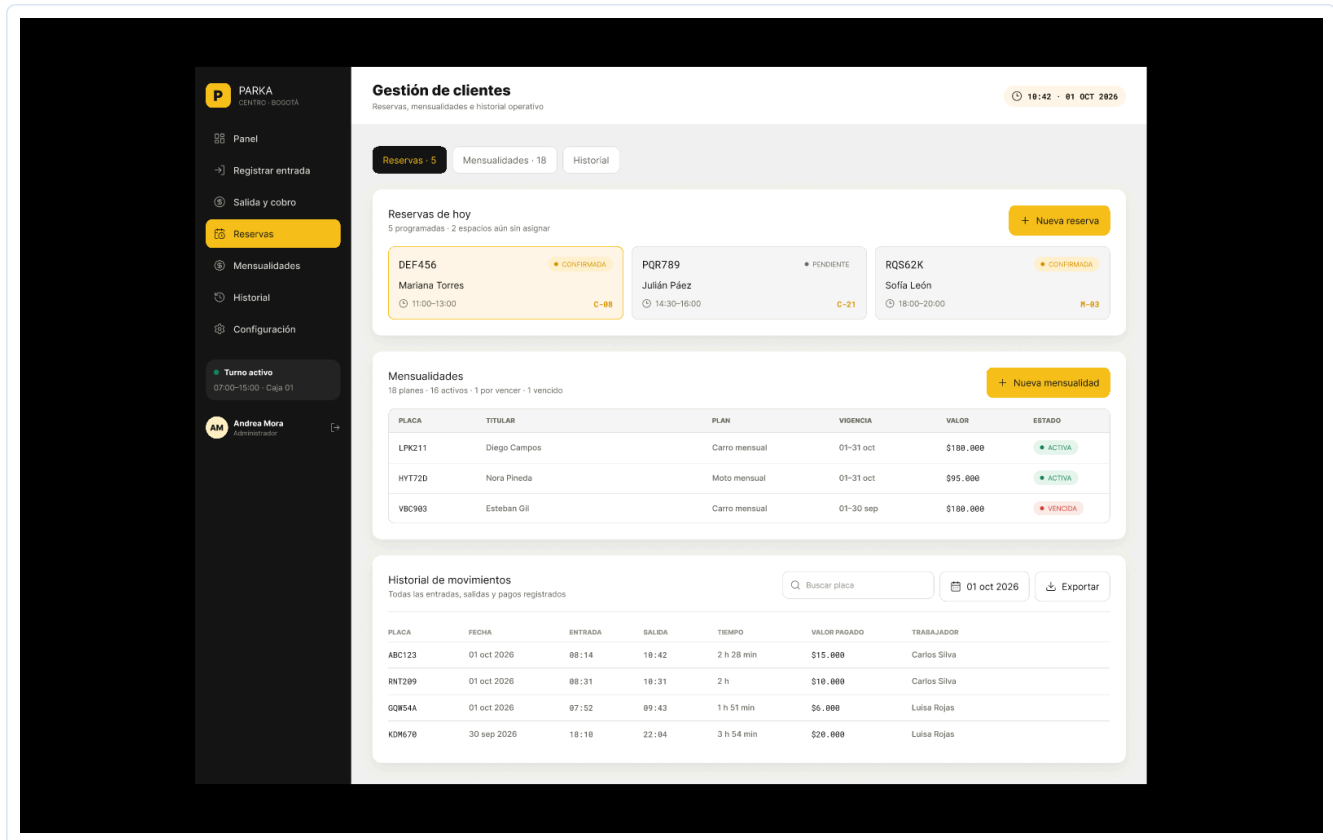


Figura 9. Pantalla F06 del prototipo – Gestión de clientes (escritorio).

10.9 Configuración (solo Administrador)

1. Pulse **Configuración** en el menú (no aparece para el Trabajador).
2. En **Tarifas vigentes**, modifique los valores de Carro y Moto para Hora, Fracción, Noche, Día, Reserva y Mensualidad.
3. Pulse **Guardar cambios**.
4. Consulte la matriz de **Permisos por rol**.

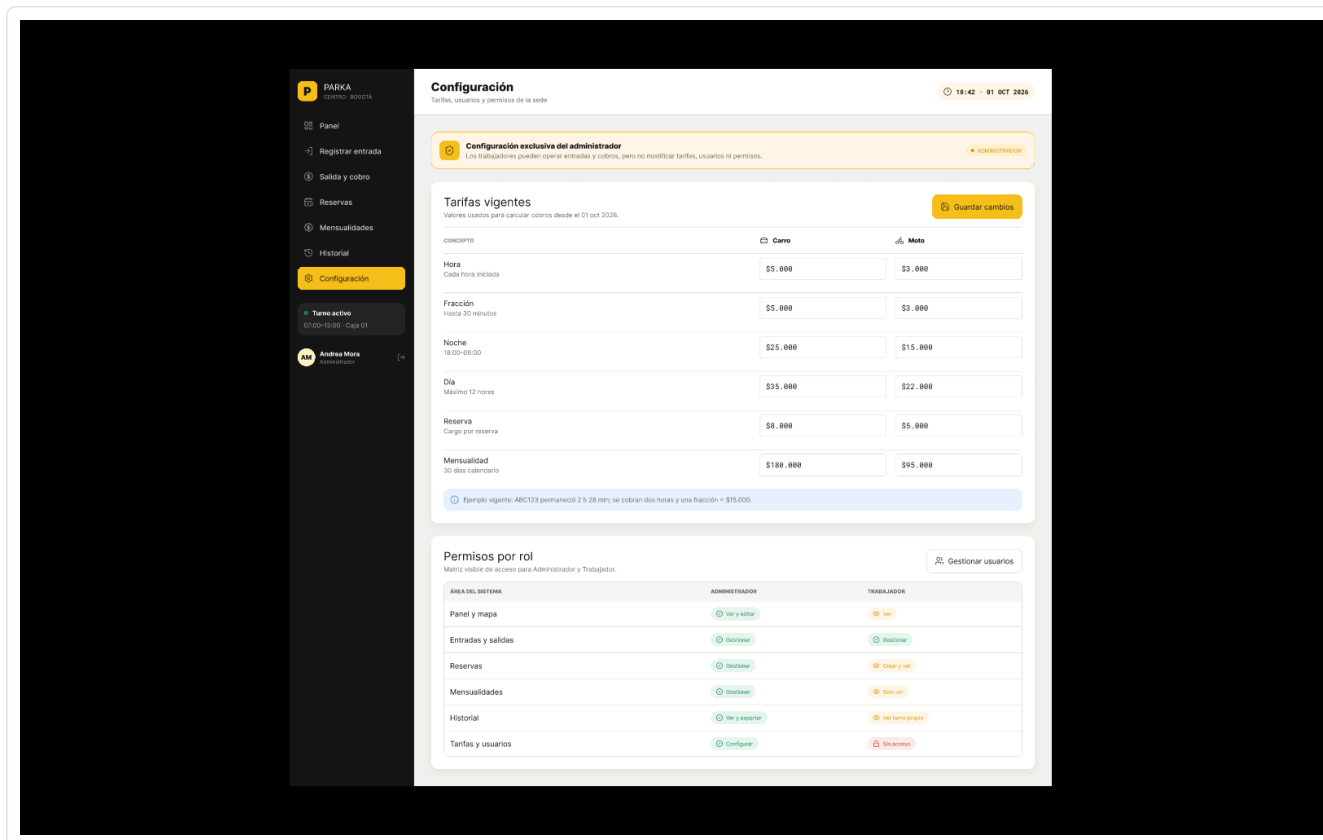


Figura 10. Pantalla F07 del prototipo – Configuración (escritorio, exclusiva del Administrador).

10.10 Gestión de usuarios (solo Administrador)

En Configuración, pulse **Gestionar usuarios**. La pantalla de gestión de usuarios está pendiente de diseño en el prototipo.

10.11 Uso desde el teléfono

La versión móvil incluye inicio de sesión, panel, registro de entrada y salida con cobro, con una barra inferior: **Panel, Entrada, Salida y Más**.

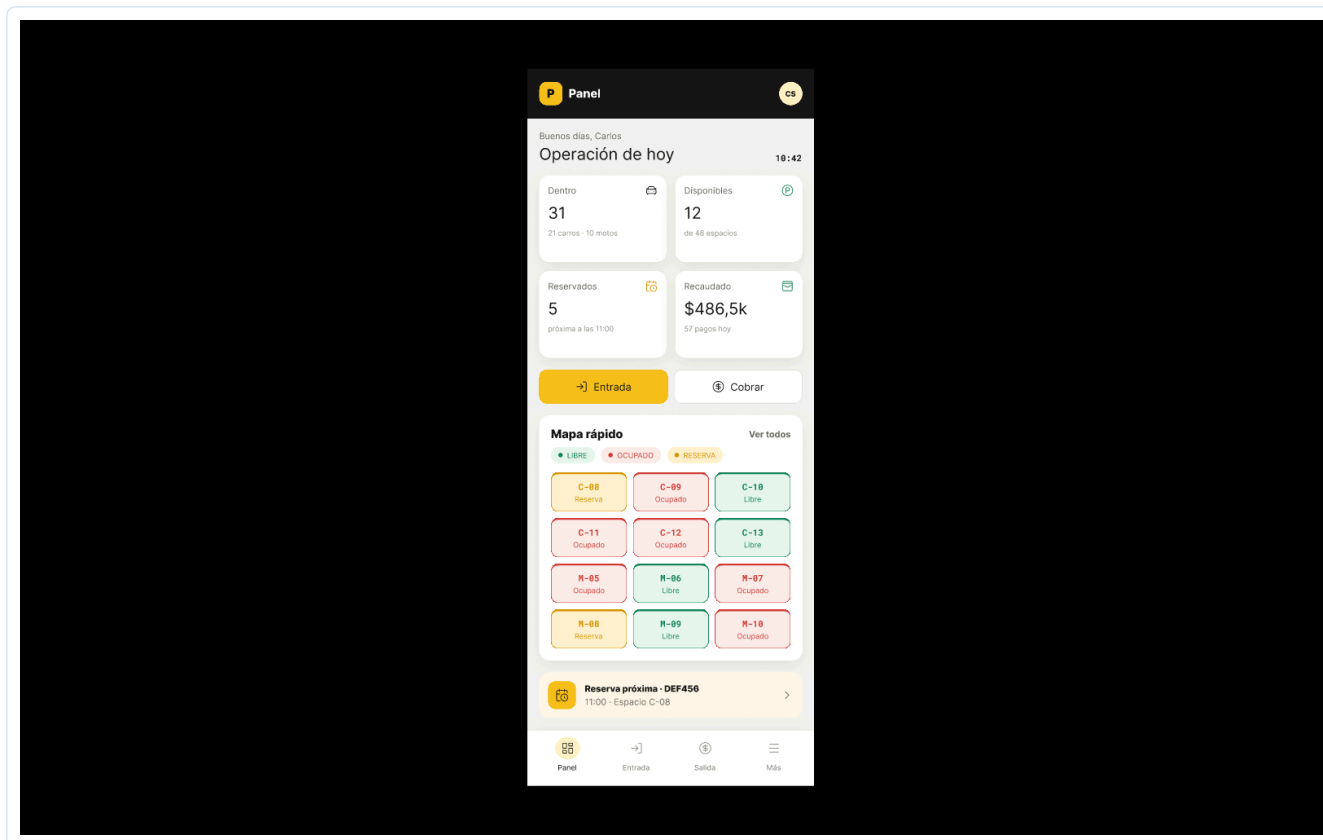


Figura 11. Pantalla F09 del prototipo – Panel (móvil, rol Trabajador).

10.12 Mensajes del sistema (según Figma)

Mensaje	Significado
Formato validado · sin coincidencias activas	La placa tiene formato correcto y no tiene una entrada abierta.
C-12 está disponible · Se marcará como ocupado al guardar	El espacio seleccionado puede usarse.
Encontrado	La placa buscada tiene una entrada registrada.
Monto suficiente	El efectivo cubre el total.
Con \$12.000, faltan \$3.000 para completar	El efectivo no alcanza; se indica el faltante.
Al confirmar... el espacio cambiará de ocupado a disponible	Resumen previo a registrar la salida.
Configuración exclusiva del administrador	El Trabajador no puede modificar tarifas, usuarios ni permisos.

10.13 Cierre de sesión

Pulse el ícono de salida junto a su nombre en la parte inferior del menú lateral.

11. Estado actual y trabajo pendiente

Componente	Estado	Detalle verificado
Diseño funcional (Figma)	COMPLETO	11 pantallas que cubren 21 funcionalidades (RF-001 a RF-021).
Frontend	EXISTENTE EN CÓDIGO	Plantilla base de Vite + React con configuración PWA (manifest, service worker e íconos). No contiene pantallas del parqueadero.
Pantallas de PARKINLOCK	DISEÑADO EN FIGMA – PENDIENTE DE IMPLEMENTACIÓN	Ninguna de las 11 pantallas está construida en el código.
Backend / API	NO INICIADO	La carpeta backend-Parkinlock está vacía y no está registrada en Git.
Base de datos	PROPUESTO – PENDIENTE DE IMPLEMENTACIÓN	No existe en el código. Se propone un modelo MySQL (documentos 05 y 06).
Despliegue	NO REALIZADO	No hay evidencia de despliegue.

Indicador	Cantidad
Funcionalidades diseñadas en Figma	21
Funcionalidades implementadas en código	0
Funcionalidades pendientes de implementación	21

11.1 Partes diseñadas

Las 21 funcionalidades (RF-001 a RF-021) en 11 pantallas de Figma para escritorio y móvil, incluida la matriz de permisos por rol.

11.2 Partes implementadas

Base técnica del frontend: proyecto React + Vite, configuración PWA (manifest, service worker con actualización automática, íconos generados), linter oxlint y repositorio en GitHub. **Ninguna funcionalidad del parqueadero está implementada.**

11.3 Trabajo pendiente

#	Pendiente	Requisitos
1	Definir la tecnología del backend y crear su estructura.	—
2	Crear la base de datos MySQL con el script del documento 06.	Todos
3	Implementar autenticación, roles y permisos.	RF-001 a RF-004, RF-018
4	Construir el panel, el mapa y los movimientos.	RF-005 a RF-007, RF-019 a RF-021
5	Construir entrada, salida, cobro y pago.	RF-008 a RF-012
6	Construir reservas, mensualidades e historial con exportación.	RF-013 a RF-016
7	Construir la configuración de tarifas.	RF-017
8	Diseñar las pantallas faltantes: recuperación de contraseña, nueva reserva, nueva mensualidad, gestión de usuarios.	RF-002, RF-013, RF-014, RF-018

#	Pendiente	Requisitos
9	Reemplazar favicon.svg por el logo oficial y ajustar colores del manifest.	RNF-005
10	Desplegar la aplicación.	RNF-014

12. Conclusiones

- PARKINLOCK cuenta con un diseño funcional completo en Figma que define 21 funcionalidades para Administrador y Trabajador, cubriendo todo el ciclo de atención: entrada, cobro, pago, salida y liberación del espacio, además de reservas, mensualidades, historial y configuración.
- El código actual corresponde a la base técnica del frontend (React, Vite y PWA). Ninguna funcionalidad del parqueadero está implementada, el backend no se ha iniciado y no existe base de datos.
- Esta documentación aporta los artefactos necesarios para iniciar el desarrollo: 21 requisitos funcionales, 21 requisitos no funcionales, 21 historias de usuario, un modelo entidad-relación de 14 entidades y un script MySQL propuesto.
- Las decisiones pendientes más importantes son la tecnología del backend, la validación de las reglas de cobro propuestas (RC-02 a RC-06) y el diseño de las pantallas que hoy solo aparecen como botones.
- La documentación deberá actualizarse a medida que se implementen las funcionalidades, cambiando su estado de "diseñado" a "implementado".